Ước lượng cao độ F_0 kiểu Praat trên DE10-Standard

Từ file WAV 16 kHz qua HPS, FPGA và trở về HPS, so với Praat 7.0.03 trên laptop
(Intel Quartus Prime — hai đường: IP Catalog và HDL tự viết)

Bản v3 — viết lại theo đề cương chốt v2 (10/11/2026)

Mục lục

Chương 0. Bài toán, mục tiêu so sánh, và Praat thật sự làm gì **3**

0.1. Một file, hai câu trả lời 3
0.2. Ý tưởng của Boersma: Chuẩn hóa bằng tự tương quan của cửa sổ 5
0.3. Tám bước của Praat đúng như mã nguồn và các lựa chọn thiết kế phần cứng 7
0.4. Kiến trúc hệ thống và bản đồ các chương 11
0.5. Các phép đo đánh giá: GPE, VDE, FFE và sai số cent 13
0.6. Ngân sách chu kỳ, hệ số thời gian thực RTF, và mạch phát cặp mẫu phần cứng 15
0.1 Bản đồ kiểm chứng ba tầng và bộ golden v2 19

Chương 0. Bài toán, mục tiêu so sánh, và Praat thật sự làm gì

▶ MỤC TIÊU HỌC TẬP

- Nắm vững bản chất bài toán ước lượng tần số cơ bản (F_0) và lý do vì sao hai phương pháp trực giác (đếm điểm qua không và tìm đỉnh phổ FFT) thất bại trên tín hiệu tiếng nói thực tế.
- Hiểu rõ nguyên lý cốt lõi của Boersma: chuẩn hóa hàm tự tương quan bằng cách chia cho hàm tự tương quan của cửa sổ Hanning để triệt tiêu độ suy giảm giả tạo ở các độ trễ lớn.
- Nắm tường tận 8 bước thuật toán trong mã nguồn Praat 7.0.03 cùng 10 tham số chuẩn, đối chiếu chính xác giữa các khâu phần mềm của Praat và các khối xử lý phần cứng trên FPGA.
- Hiểu rõ kiến trúc toàn hệ thống: luồng dữ liệu từ tệp WAV qua HPS, cầu nối Lightweight HPS-to-FPGA, lõi tính toán trên FPGA, và đường trả kết quả F0 về HPS để ghi tệp nhị phân.
- Nắm chắc định nghĩa 4 chỉ số sai số chuẩn trong so sánh cao độ: VDE, GPE, FFE và sai số cent mịn, cùng cách tính tay trên từng khung.
- Phân tích ngân sách chu kỳ từ nguyên lý đầu tiên: 50 MHz, 3.125 chu kỳ/mẫu, 500.000 chu kỳ/bước nhảy 10 ms; dẫn xuất chính xác 98.540 phép nhân-cộng MAC và đo thực tế 98.543 chu kỳ/khung, từ đó xác lập hệ số thời gian thực (RTF).
- Hiểu kiến trúc kiểm chứng ba tầng và vai trò của bộ mô hình tương đương (twin) số nguyên `golden_v2/fixed_twin.py` làm đặc tả phần cứng cho toàn bộ dự án.

0.1. Một file, hai câu trả lời

Một tệp âm thanh WAV chuẩn 16 kHz, kênh đơn (mono), định dạng mẫu PCM 16-bit có dấu được đưa vào hệ thống. Trên máy tính cá nhân, phần mềm Praat đọc tệp âm thanh này và xuất ra một đường cao độ F_0 với chu kỳ phân tích 10 ms. Trên bo mạch DE10-Standard, hệ thống vi xử lý HPS (Hard Processor System, Cortex-A9 lõi kép) đọc cùng tệp WAV đó từ thẻ nhớ SD, truyền các mẫu âm thanh qua cầu nối bus sang vùng logic FPGA (Cyclone V 5CSXFC6D6F31C6N). Lõi phần cứng trên FPGA tính toán cao độ từng khung và trả kết quả về HPS dưới dạng tệp nhị phân `pitch_f0.bin` ở định dạng số cố định Q12.4.

Câu hỏi trung tâm mà anh Tú cùng bất kỳ kỹ sư phần cứng nào cũng phải đối mặt là: hai đường cong cao độ đó lệch nhau bao nhiêu, và nguyên nhân vật lý hay số học nào tạo ra sự chênh lệch ở từng mili-giây? Để trả lời thấu đáo câu hỏi này, trước hết ta phải hiểu vì sao việc ước lượng cao độ tiếng nói lại khó, và vì sao những cách tiếp cận trực giác thông thường đều sụp đổ khi áp dụng vào thực tế.

Sự sụp đổ của phương pháp đếm điểm qua không (Zero-Crossing Counting)

Phương pháp đơn giản nhất mà người mới học xử lý tín hiệu thường nghĩ đến là đếm số lần tín hiệu đổi dấu qua mức 0 (zero-crossing) trong một khoảng thời gian, từ đó suy ra tần số. Tuy nhiên, một sự thật vật lý cơ bản là: tiếng nói có thanh (voiced speech) của con người hầu như không bao giờ là một sóng hình sin đơn âm. Khi dây thanh âm rung động, luồng khí đóng mở theo chu kỳ tạo ra dạng sóng có chứa rất nhiều hài âm (harmonics). Nếu hài âm bậc hai ($2f_0$) có biên độ mạnh và lệch pha so với sóng cơ bản, nó sẽ uốn dạng sóng tổng hợp cắt qua trục 0 nhiều lần trong cùng một chu kỳ lặp lại.

Để thấy rõ sự thất bại này bằng số liệu định lượng, ta xét một đoạn tín hiệu tổng hợp được tạo bởi kịch bản kiểm chứng `verify_v3/ch00_naive_methods.py`:

$$x(t) = \sin(2\pi \cdot 100t) - 0{,}75\cos(2\pi \cdot 200t) \tag{1}$$

Tần số cơ bản thực sự là $f_0 = 100{,}0$ Hz, tần số lấy mẫu $f_s = 16.000$ Hz. Một chu kỳ cơ bản kéo dài $T_0 = 1/100 = 10$ ms, tương ứng đúng $P = 160$ mẫu. Nếu tín hiệu là sóng sin thuần, một chu kỳ 160 mẫu sẽ cắt qua trục 0 đúng 2 lần. Nhưng khi kiểm tra trên đoạn tín hiệu dài 3 chu kỳ (480 mẫu), bộ đếm điểm qua không ghi nhận tổng cộng 12 lần đổi dấu, tức trung bình 4 điểm qua không trong mỗi chu kỳ. Tần số ước lượng bị thổi phồng thành:

$$f_{\text{est}} = \frac{12}{2 \times 3 \times 0{,}01\ \text{s}} = 200{,}0\ \text{Hz} \tag{2}$$

Thuật toán bị đánh lừa hoàn toàn bởi hài âm bậc hai, báo sai lệch cao độ gấp đôi (100% lỗi) ngay trên một tín hiệu hoàn toàn tuần hoàn và không hề có nhiễu bên ngoài.

Ví dụ tính tay thu nhỏ. Để nhìn thấy từng bước ở mức mẫu, xét 16 mẫu rời rạc đại diện cho một chu kỳ rút gọn (lấy trực tiếp từ log của `verify_v3/ch00_naive_methods.py`):

n	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
$x[n]$	0	-18	-9	36	100	149	151	95	0	-95	-151	-149	-100	-36	9	18

Quan sát dãy mẫu: từ $n = 2$ (giá trị -9) sang $n = 3$ ($+36$), tín hiệu đổi dấu lần thứ nhất. Đến $n = 8$, tín hiệu bằng 0 và giảm xuống -95 ở $n = 9$, đổi dấu lần thứ hai. Từ $n = 13$ (-36) sang $n = 14$ ($+9$), tín hiệu lại đổi dấu lần thứ ba. Cùng với điểm 0 ban đầu ở $n = 0$, ta có 4 lần cắt qua 0 trong chỉ một chu kỳ cơ bản. Mạch đếm phần cứng nếu dựa vào hiện tượng này sẽ nhận diện nhầm chu kỳ thành 8 mẫu thay vì 16 mẫu.

Sự sụp đổ của phương pháp chọn đỉnh phổ FFT (FFT Peak Picking)

Cách tiếp cận thứ hai là chuyển tín hiệu sang miền tần số bằng phép biến đổi Fourier nhanh (FFT, Fast Fourier Transform) rồi tìm vị trí bin có biên độ cực đại. Để bám sát ngữ điệu biến đổi liên tục của tiếng nói, ta bắt buộc phải phân tích trên từng khung thời gian ngắn khoảng vài chục mili-giây (thường từ 20 ms đến 40 ms). Tại tần số lấy mẫu $f_s = 16.000$ Hz, một khung 40 ms tương ứng với 640 mẫu (hoặc 638 mẫu theo quy ước làm tròn đối xứng của Praat).

Độ phân giải tần số của phép phân tích FFT bị chặn cứng bởi chiều dài khung N:

$$\Delta f = \frac{f_s}{N} = \frac{16.000}{640} = 25{,}00 \text{ Hz} \quad (\text{với } N = 638, \Delta f = \frac{16.000}{638} \approx 25{,}08 \text{ Hz}) \tag{3}$$

Khoảng cách giữa hai vạch phổ kề nhau lên tới 25 Hz. Hãy xét một giọng nam trầm có tần số cơ bản thực là $f_0 = 112{,}5$ Hz. Tần số này rơi vào chính giữa bin 4 ($4 \times 25 = 100$ Hz) và bin 5 ($5 \times 25 = 125$ Hz). Bộ chọn đỉnh phổ dù hoàn hảo nhất cũng chỉ có thể chỉ định bin 4 hoặc bin 5, gây ra sai số tuyệt đối ít nhất là $12{,}5$ Hz.

Trong âm nhạc học và ngữ âm học, tai người cảm nhận cao độ theo hàm logarit của tần số. Đơn vị đo độ chênh lệch cao độ chuẩn là **cent**, trong đó 1 quãng tám (tần số gấp đôi) tương ứng với 1.200 cent, và 1 nửa cung (semitone) tương ứng với 100 cent:

$$\Delta\text{cent} = 1.200 \cdot \left| \log_2 \left(\frac{f_{\text{est}}}{f_0} \right) \right| \tag{4}$$

Áp dụng cho trường hợp chọn nhầm bin 100 Hz khi tần số thực là 112,5 Hz:

$$\Delta\text{cent} = 1.200 \cdot \left| \log_2 \left(\frac{100{,}0}{112{,}5} \right) \right| \approx 203{,}91 \text{ cent} \tag{5}$$

(kết quả được kiểm chứng trong `verify_v3/ch00_naive_methods.log`). Sai số hơn 200 cent tương đương với hai nửa cung (một cung nguyên) trong âm nhạc. Một bộ ước lượng F_0 với sai số thô như vậy hoàn toàn vô giá trị trong các ứng dụng phân tích ngữ điệu hoặc đối sánh khoa học. Muốn giảm Δf xuống dưới 1 Hz, FFT cần cửa sổ dài hơn 1 giây, nhưng trong 1 giây đó người nói đã phát âm qua 3 đến 5 âm tiết khác nhau, tính dừng của tín hiệu bị phá vỡ hoàn toàn.

Bản chất của phép tự tương quan (Autocorrelation)

Để khắc phục triệt để hai nhược điểm trên, ta quay về nguyên lý đầu tiên: **một tín hiệu tuần hoàn luôn có hình dạng giống hệt chính nó khi bị dịch trễ đi đúng một chu kỳ cơ bản**. Phép đo mức độ tương đồng giữa tín hiệu $x[n]$ và bản sao dịch trễ của nó $x[n + \tau]$ ở độ trễ τ (lag) được định nghĩa qua hàm tự tương quan ngắn hạn:

$$R[\tau] = \sum_{n=0}^{N-1-\tau} x[n]x[n+\tau] \tag{6}$$

Ví dụ tính tay số nguyên thu nhỏ. Xét một tín hiệu rời rạc tuần hoàn hoàn hảo với chu kỳ cơ bản $P = 3$ mẫu, độ dài khung $N = 9$ mẫu:

$$x = [3, -1, -2,\ 3, -1, -2,\ 3, -1, -2] \tag{7}$$

Ta tính trực tiếp giá trị tích vô hướng $R[\tau]$ theo từng bước cho các độ trễ từ $\tau = 0$ đến $\tau = 5$ (trích xuất từ kịch bản kiểm chứng `verify_v3/ch00_naive_methods.py`):

- Tại $\tau = 0$: lấy 9 cặp mẫu tự nhân với chính nó:

$$\begin{aligned} R[0] &= 3^2 + (-1)^2 + (-2)^2 + 3^2 + (-1)^2 + (-2)^2 + 3^2 + (-1)^2 + (-2)^2 \\ &= 9 + 1 + 4 + 9 + 1 + 4 + 9 + 1 + 4 = 42 \end{aligned}$$

- Tại $\tau = 1$: nhân 8 cặp mẫu kề nhau:

$$\begin{aligned} R[1] &= 3(-1) + (-1)(-2) + (-2)3 + 3(-1) + (-1)(-2) + (-2)3 + 3(-1) + (-1)(-2) \\ &= -3 + 2 - 6 - 3 + 2 - 6 - 3 + 2 = -15 \end{aligned}$$

- Tại $\tau = 2$: nhân 7 cặp mẫu cách nhau 2 vị trí:

$$\begin{aligned} R[2] &= 3(-2) + (-1)3 + (-2)(-1) + 3(-2) + (-1)3 + (-2)(-1) + 3(-2) \\ &= -6 - 3 + 2 - 6 - 3 + 2 - 6 = -20 \end{aligned}$$

- Tại $\tau = 3$: nhân 6 cặp mẫu cách nhau đúng một chu kỳ $P = 3$:

$$\begin{aligned} R[3] &= 3(3) + (-1)(-1) + (-2)(-2) + 3(3) + (-1)(-1) + (-2)(-2) \\ &= 9 + 1 + 4 + 9 + 1 + 4 = 28 \end{aligned}$$

- Tại $\tau = 4$: nhân 5 cặp mẫu, thu được $R[4] = -8$.
- Tại $\tau = 5$: nhân 4 cặp mẫu, thu được $R[5] = -13$.

Chỗ dễ vướng: Vì sao $R[3] = 28$ lại bé hơn $R[0] = 42$ dù tín hiệu tuần hoàn tuyệt đối? Đây là bẫy tư duy kinh điển mà người thiết kế phần cứng hay gặp. Ở độ trễ $\tau = 3$, số cặp mẫu còn lại để nhân với nhau chỉ là $N - \tau = 9 - 3 = 6$ cặp, trong khi ở $\tau = 0$ ta cộng dồn tới 9 cặp. Nếu ta tính giá trị trung bình không chệch bằng cách chia tổng tích lũy cho số cặp mẫu tham gia:

$$\frac{R[0]}{9 - 0} = \frac{42}{9} \approx 4{,}67 \tag{8}$$

$$\frac{R[3]}{9 - 3} = \frac{28}{6} \approx 4{,}67 \tag{9}$$

Hai con số hoàn toàn trùng khớp! Đỉnh tương quan tại $\tau = 3$ đã chỉ ra chính xác chu kỳ cơ bản của tín hiệu là 3 mẫu mà không bị nhiễu hài âm đánh lừa như phương pháp đếm điểm qua 0, cũng không bị lượng tử hóa thô thiển như phương pháp chọn đỉnh FFT.

Tóm lại: Đếm điểm qua không thất bại vì hài âm bậc hai tạo thêm điểm cắt giả, lấy đỉnh FFT thất bại vì độ phân giải bin thô tới 204 cent ở 100 Hz; phương pháp tự tương quan đo mức độ tương đồng theo miền thời gian, mở ra con đường xác định chu kỳ cơ bản chính xác và bền vững trước cấu trúc hài âm phức tạp của tiếng nói.

0.2. Ý tưởng của Boersma: Chuẩn hóa bằng tự tương quan của cửa sổ

Khi áp dụng phương pháp tự tương quan lên các đoạn tiếng nói thực tế, một bài toán kỹ thuật mới lập tức nảy sinh: ta không thể tính toán trên toàn bộ tệp âm thanh dài hàng phút mà bắt buộc phải cắt tín hiệu thành từng khung ngắn hữu hạn (khoảng 40 ms) để theo dõi sự biến đổi cao độ theo thời gian. Việc cắt khung bằng một hàm chữ nhật đột ngột (rectangular window) sẽ tạo ra các điểm gián đoạn sắc nhọn ở hai biên khung, dẫn đến hiện tượng rò rỉ năng lượng phổ (spectral leakage) nghiêm trọng. Để khắc phục hiện tượng này, giải pháp kinh điển là nhân khung tín hiệu với một hàm cửa sổ êm dịu, tiêu biểu là cửa sổ Hanning (hoặc Hamming), có biên độ vuốt dần về 0 ở hai đầu mút.

Tuy nhiên, việc nhân cửa sổ lại kéo theo một hậu quả tiêu cực khác: các mẫu âm thanh nằm xa tâm khung bị suy giảm biên độ mạnh mẽ. Khi ta tính hàm tự tương quan trên tín hiệu đã nhân cửa sổ $x_w[n] = x[n] \cdot w[n]$:

$$R_{xw}[\tau] = \sum_{n=0}^{N-1-\tau} x_w[n] x_w[n+\tau] = \sum_{n=0}^{N-1-\tau} x[n] w[n] x[n+\tau] w[n+\tau] \tag{10}$$

Độ trễ τ càng lớn thì khoảng chồng lấn giữa $w[n]$ và $w[n+\tau]$ càng thu hẹp, và tích $w[n]w[n+\tau]$ càng bị đè bẹp về gần 0. Đồ thị $R_{xw}[\tau]$ xuất hiện một sườn dốc suy thoái nhân tạo theo τ. Hệ quả là đỉnh tương quan ứng với chu kỳ cơ bản thực sự T_0 của một giọng nam trầm (nằm ở các độ trễ lớn, ví dụ $\tau \approx 160$ mẫu tại 100 Hz) bị lùn đi nghiêm trọng. Nếu dùng hàm tương quan chuẩn hóa thông thường $R_{xw}[\tau]/R_{xw}[0]$, giá trị đỉnh này thường xuyên rơi xuống dưới ngưỡng hữu thanh (voicing threshold mặc định của Praat là 0,45) khiến khung âm thanh bị nhận diện nhầm thành vô thanh, hoặc bị lép vế trước một đỉnh phụ ở độ trễ nhỏ hơn ($\tau = T_0/2$), gây ra lỗi kinh điển là **nhân đôi quãng tám (octave doubling)**.

Cơ chế chia cho tự tương quan cửa sổ $R_w[\tau]$

Đóng góp đột phá của Paul Boersma (1993) nằm ở nhận định giải tích: hàm tự tương quan của tín hiệu đã nhân cửa sổ xấp xỉ bằng tích giữa hàm tự tương quan thực sự của tín hiệu $R_x[\tau]$ với hàm tự tương quan của chính hàm cửa sổ $R_w[\tau]$:

$$R_{xw}[\tau] \approx R_x[\tau] \cdot R_w[\tau] \tag{11}$$

Trong đó hàm tự tương quan của cửa sổ được định nghĩa hoàn toàn tương tự:

$$R_w[\tau] = \sum_{n=0}^{N-1-\tau} w[n]w[n+\tau] \tag{12}$$

Từ mối liên hệ nhân này, Boersma đề xuất thuật toán chuẩn hóa: muốn tìm lại hàm tự tương quan chuẩn hóa không thiên vị $r[\tau]$ của bản thân tín hiệu gốc (với giá trị lý tưởng $r[0] = 1$ và $r[T_0] = 1$ đối với tín hiệu tuần hoàn hoàn hảo), ta chỉ cần lấy $R_{xw}[\tau]$ chia cho $R_w[\tau]$, rồi chuẩn hóa tỉ lệ theo năng lượng tại $\tau = 0$:

$$r[\tau] = \frac{R_{xw}[\tau]/R_w[\tau]}{R_{xw}[0]/R_w[0]} = \frac{R_{xw}[\tau]}{R_w[\tau]} \cdot \frac{R_w[0]}{R_{xw}[0]} \tag{13}$$

Phép chia này đóng vai trò như một bộ bù độ dốc: vì $R_w[\tau]$ giảm dần về 0 khi τ tăng theo đúng quy luật hình học của cửa sổ, việc chia cho $R_w[\tau]$ sẽ nâng bổng các đỉnh ở độ trễ xa lên đúng biên độ thực sự của chúng, triệt tiêu hoàn toàn sự thiên lệch về phía các độ trễ nhỏ.

Vì sao phép xấp xỉ $R_{xw}[\tau] \approx R_x[\tau] \cdot R_w[\tau]$ lại đúng?

Để trả lời thuyết phục người kỹ sư phần cứng, ta xét bản chất vật lý của hai chuyển động: dao động nhanh của dây thanh âm và sự biến thiên chậm của đường bao cửa sổ.

Trong thuật toán Praat, chiều dài khung phân tích được ấn định bằng đúng 3 chu kỳ của tần số thấp nhất (pitch floor $f_{\text{min}} = 75$ Hz):

$$T_{\text{window}} = \frac{3}{f_{\text{min}}} = \frac{3}{75} = 0{,}04 \text{ s} = 40 \text{ ms} \quad (N = 638 \text{ mẫu tại } f_s = 16 \text{ kHz}) \tag{14}$$

Trong khi đó, tần số cơ bản của người nói dao động từ 75 Hz đến 600 Hz, tương ứng với chu kỳ cơ bản T_0 từ 26 mẫu (600 Hz) đến 213 mẫu (75 Hz). Điều này có nghĩa là bên trong một khung 638 mẫu, tín hiệu tiếng nói thực hiện từ 3 đến hơn 24 chu kỳ dao động tuần hoàn nhanh.

Đường bao cửa sổ Hanning $w[n]$ là một hàm cosin trơn dốc êm dịu trải dài trên toàn bộ 638 mẫu. Trong phạm vi một chu kỳ đơn lẻ T_0, giá trị của $w[n]$ và $w[n+\tau]$ gần như là một hằng số cục bộ. Khi ta lấy tổng tích lũy $\sum x[n]x[n+\tau] \cdot (w[n]w[n+\tau])$, đại lượng dao động nhanh $x[n]x[n+\tau]$ tạo ra giá trị kỳ vọng trung bình tuần hoàn $R_x[\tau]$, trong khi đại lượng biến thiên chậm $w[n]w[n+\tau]$ có thể đưa ra ngoài dấu tích phân cục bộ dưới dạng trọng số bao $R_w[\tau]$. Phép nhân chập giữa phổ tín hiệu hẹp và phổ cửa sổ rộng trong miền tần số tương ứng với phép nhân đường bao trực tiếp trong miền thời gian.

Ví dụ tính tay thu nhỏ: Sóng sin thuần qua cửa sổ Hanning 8 mẫu

Để minh chứng tường minh bằng số học cụ thể, ta xét một thí nghiệm mô phỏng thu nhỏ được thực thi bởi kịch bản `verify_v3/ch00_norm_sine.py`. Ta sử dụng một sóng sin tuần hoàn hoàn hảo có chu kỳ $P = 4$ mẫu (tương ứng tần số $f_0 = f_s/4 = 4.000$ Hz), chiều dài phân tích $N = 8$ mẫu:

$$x[n] = \sin\left(\frac{2\pi n}{4}\right) = [0{,}000,\ 1{,}000,\ 0{,}000,\ -1{,}000,\ 0{,}000,\ 1{,}000,\ 0{,}000,\ -1{,}000] \tag{15}$$

Hàm cửa sổ Hanning 8 mẫu của Praat tuân thủ công thức $N+1$ (được định nghĩa trong mã nguồn Praat `Sound_to_Pitch.cpp` dòng 423–424):

$$w[i] = 0{,}5 - 0{,}5\cos\left(\frac{2\pi i}{N+1}\right), \quad i = 1, \ldots, N \quad (i = n+1) \tag{16}$$

Với $N = 8$, các mẫu cửa sổ tính ra là:

$$w = [0{,}116978,\ 0{,}413176,\ 0{,}750000,\ 0{,}969846,\ 0{,}969846,\ 0{,}750000,\ 0{,}413176,\ 0{,}116978]$$

Tín hiệu sau khi nhân cửa sổ $x_w[n] = x[n] \cdot w[n]$ là:

$$x_w = [0{,}0000,\ 0{,}4132,\ 0{,}0000,\ -0{,}9698,\ -0{,}0000,\ 0{,}7500,\ 0{,}0000,\ -0{,}1170]$$

Bảng 1 liệt kê kết quả tính tay chi tiết hàm tự tương quan của cửa sổ $R_w[\tau]$, hàm tự tương quan của tín hiệu nhân cửa sổ $R_{xw}[\tau]$, hệ số tương quan chưa chuẩn hóa $r_{\text{unnorm}}[\tau] = R_{xw}[\tau]/R_{xw}[0]$, và hệ số chuẩn hóa Boersma $r_{\text{Boersma}}[\tau]$ cho các độ trễ từ $\tau = 0$ đến $\tau = 4$ (trích xuất nguyên văn từ `verify_v3/ch00_norm_sine.log`).

Bảng 1: Đối chiếu giá trị tự tương quan trước và sau khi chuẩn hóa cửa sổ Boersma trên sóng sin thuần ($N = 8, P = 4$)

Độ trễ τ	$R_w[\tau]$	$R_{xw}[\tau]$	$r_{\text{unnorm}}[\tau]$	$r_{\text{Boersma}}[\tau]$	Đánh giá vật lý
0	3,375000	1,687500	1,000000	1,000000	Gốc chuẩn hóa năng lượng
1	3,111800	0,000000	0,000000	0,000000	Trực giao ở góc pha $90°$
2	2,431670	$-1{,}215835$	$-0{,}720495$	$-1{,}000000$	Ngược pha hoàn toàn ($180°$)
3	1,590835	0,000000	0,000000	0,000000	Trực giao ở góc pha $270°$
4	**0,846665**	**0,423332**	**0,250864**	**1,000000**	**Chu kỳ cơ bản** $P = 4$

Đọc bảng và kết luận định lượng. Hãy nhìn thẳng vào dòng $\tau = 4$, chính là chu kỳ lặp lại thực tế của sóng sin:

- Nếu **không chuẩn hóa cửa sổ**, hệ số tương quan chỉ đạt $r_{\text{unnorm}}[4] = 0{,}250864 \approx 0{,}251$. Tín hiệu thuần nhất 100% tuần hoàn nhưng giá trị đỉnh tương quan bị sụt giảm tới gần 75% năng lượng! Với ngưỡng voicing mặc định của Praat là 0,45, thuật toán sẽ kết luận ngay khung này là *vô thanh* (unvoiced) do $0{,}251 < 0{,}45$.
- Ngược lại, khi áp dụng **chuẩn hóa Boersma**: ta lấy $R_{xw}[4]/R_w[4] = 0{,}423332/0{,}846665 = 0{,}500000$. Chia cho tỉ số tại gốc $R_{xw}[0]/R_w[0] = 1{,}687500/3{,}375000 = 0{,}500000$, ta thu được kết quả chính xác tuyệt đối:

$$r_{\text{Boersma}}[4] = \frac{0{,}500000}{0{,}500000} = 1{,}000000 \tag{17}$$

Giá trị 1,000000 phản ánh đúng 100% tính tuần hoàn của sóng sin, không hề bị ảnh hưởng bởi việc cửa sổ bị thót hẹp ở hai đầu. Đây chính là linh hồn toán học giải thích vì sao phần mềm Praat đạt được độ tin cậy và danh tiếng vượt trội trong cộng đồng nghiên cứu ngữ âm học suốt ba thập kỷ qua.

Tóm lại: Cửa sổ cắt khung làm suy hao giả tạo năng lượng ở các độ trễ lớn khiến đỉnh cao độ thật bị lùn đi; ý tưởng cốt lõi của Boersma là chia hàm tự tương quan của tín hiệu cho hàm tự tương quan của cửa sổ $R_{xw}[\tau]/R_w[\tau]$, khôi phục trọn vẹn độ mạnh của chu kỳ cơ bản và triệt tiêu định kiến thiên lệch về các độ trễ nhỏ.

0.3. Tám bước của Praat đúng như mã nguồn và các lựa chọn thiết kế phần cứng

Mọi so sánh định lượng giữa bộ đo trên FPGA và phần mềm Praat chỉ có giá trị khoa học khi và chỉ khi ta hiểu chính xác từng dòng mã nguồn của Praat thực sự làm gì. Trong các tài liệu sơ bộ trước đây, thuật toán Praat thường được mô tả qua trí nhớ hoặc dựa trên các khái niệm chung chung, dẫn đến nhiều sai lệch nghiêm trọng về kích thước cửa sổ, số lượng tham số, hay phạm vi tính toán. Ở đây, ta đối chiếu trực tiếp với mã nguồn chuẩn của bản phát hành **Praat 7.0.03** (phát hành ngày 08/11/2026, ghim làm mốc tham chiếu cho toàn bộ tài liệu), cụ thể là các tệp `fon/Sound_to_Pitch.cpp`, `fon/Pitch.cpp`, `fon/Sampled.cpp` và `fon/praat_Sound.cpp`.

Tám bước thuật toán theo đúng thứ tự thực thi trong mã nguồn Praat

Thuật toán ước lượng cao độ bằng tự tương quan thô của Praat vận hành tuần tự qua tám bước sau:

Bước 1: Tính đỉnh biên độ toàn cục (`globalPeak`). Trong tệp `Sound_to_Pitch.cpp` (dòng 375–383), Praat quét qua toàn bộ tệp âm thanh đầu vào, tính giá trị trung bình DC của cả tệp, rồi xác định đỉnh biên độ cực đại:

$$\text{globalPeak} = \max_t |x(t) - \bar{x}_{\text{file}}| \tag{18}$$

Giá trị này phản ánh mức biên độ đỉnh tuyệt đối của người nói, được giữ nguyên làm mốc quy chiếu xuyên suốt để thiết lập ngưỡng yên lặng động cho từng khung ở Bước 8.

Bước 2: Phân bố lưới khung thời gian (`Sampled_shortTermAnalysis`). Trong tệp `Sampled.cpp` (dòng 56–70), Praat không đặt khung đầu tiên tại mẫu 0 mà căn các khung đối xứng hoàn hảo quanh tâm của tệp âm thanh. Với độ dài tệp T, số khung phân tích được tính bởi công thức làm tròn sàn $\lfloor (T - 0{,}04)/0{,}01 \rfloor + 1$. Vị trí tâm của khung đầu tiên t_1 phụ thuộc vào độ dài tổng thể của tệp. Vì vậy, khung thứ nhất không bắt đầu ở mẫu 0 mà lệch đi một số mẫu nhất định; lõi phần cứng FPGA bắt buộc phải nhận tọa độ t_1 và chỉ số mẫu bắt đầu do bộ vi xử lý HPS tính toán để đảm bảo hai bên phân tích trên đúng từng mẫu âm thanh.

Bước 3: Trừ trung bình cục bộ trên 426 mẫu. Trong tệp `Sound_to_Pitch.cpp` (dòng 56–66), Praat tính giá trị trung bình cục bộ của tín hiệu trên đoạn dài đúng hai chu kỳ dài nhất về hai phía tâm khung ("one longest period to both sides"). Tại $f_{\text{min}} = 75$ Hz, chu kỳ dài nhất là $T_{0,\text{max}} = 16.000/75 = 213{,}33$ mẫu, làm tròn thành 213 mẫu. Do đó, trung bình cục bộ được tính trên đúng $2 \times 213 = 426$ mẫu quanh tâm khung ($106 \ldots 531$), hoàn toàn không phải trên toàn bộ 638 mẫu. Giá trị trung bình này được trừ khỏi khung để triệt tiêu triệt để thành phần một chiều (DC offset) cục bộ, ngăn ngừa sai lệch hàm tự tương quan.

Bước 4: Nhân cửa sổ Hanning $N + 1$ với $N = 638$ mẫu. Trong tệp `Sound_to_Pitch.cpp` (dòng 335–339 và 423–424), kích thước cửa sổ phân tích được làm tròn đối xứng: từ 640 mẫu lý thuyết, Praat lấy nửa cửa sổ half $= 640/2 - 1 = 319$ mẫu, suy ra chiều dài cửa sổ thực tế là $N = 2 \times 319 = 638$ mẫu. Tín hiệu sau khi trừ trung bình cục bộ được nhân với hàm cửa sổ Hanning dạng $N + 1$:

$$w[i] = 0{,}5 - 0{,}5 \cos\left(\frac{2\pi i}{639}\right), \quad i = 1, \ldots, 638 \tag{19}$$

Bước 5: Tính đỉnh biên độ cục bộ (`localPeak`). Trong tệp `Sound_to_Pitch.cpp` (dòng 430–435), Praat tìm giá trị biên độ tuyệt đối lớn nhất của tín hiệu đã nhân cửa sổ bên trong khoảng cách ± 107 mẫu quanh tâm khung:

$$\text{localPeak} = \max_{j \in [212, 425]} |x_w[j]| \tag{20}$$

Tỉ số cường độ tương đối được xác lập qua đại lượng intensity $=$ localPeak/globalPeak, phục vụ trực tiếp cho việc xác định ngưỡng chuyển đổi có thanh/vô thanh.

Bước 6: Tự tương quan và chuẩn hóa Boersma bằng FFT 1024 điểm. Trong tệp `Sound_to_Pitch.cpp` (dòng 440–450), Praat tính hàm $R_{xw}[\tau]$ bằng FFT 1024 điểm (vì độ dài nhân chập $638 \times 1{,}5 = 957$ mẫu, lũy thừa 2 nhỏ nhất là 1024). Sau đó, kết quả được chia cho $R_w[\tau]$ và chuẩn hóa về $r[0] = 1$. Mảng $r[\tau]$ được tính tới độ trễ 319 mẫu (biến `brent_ixmax` = 319) làm dữ liệu cho phép nội suy sinc.

Bước 7: Tìm kiếm ứng viên cao độ và tinh chỉnh sinc70 + Brent. Trong tệp `Sound_to_Pitch.cpp` (dòng 254–270), các đỉnh cực đại cục bộ thỏa mãn đồng thời ba điều kiện: $r[i] > r[i-1]$, $r[i] \geq r[i+1]$, và $r[i] > 0{,}225$ với $i \in [2, \text{maximumLag} - 1]$. Với $N = 638$, maximumLag $= \lfloor 638/3 \rfloor + 2 = 214$, nên chỉ số đỉnh i được rà soát trong đoạn $[27, 213]$ (độ trễ 26 và 214 đóng vai trò làm hàng xóm biên). Vị trí đỉnh ban đầu được ước lượng bằng nội suy parabol, nhưng **luôn luôn được tinh chỉnh chính xác bằng phép nội suy hàm sinc 70 điểm kết hợp thuật toán tìm kiếm Brent**, bất kể chế độ `Very accurate` bật hay tắt. Độ mạnh (`strength`) của đỉnh nếu vượt quá 1 sẽ được phản xạ thành $1/\text{strength}$. Praat lưu lại tối đa 15 ứng viên có độ mạnh lớn nhất trong mỗi khung.

Bước 8: Thuật toán quy hoạch động Viterbi (Path Finder). Trong tệp `Pitch.cpp` (dòng 553–560), một bộ tìm đường tối ưu toàn cục (Viterbi) xâu chuỗi các ứng viên qua tất cả các khung trong tệp âm thanh. Ứng viên vô thanh có độ mạnh phụ thuộc vào cường độ biên độ:

$$\text{strength}_{\text{unvoiced}} = 0{,}45 + \max\left(0,\ 2 - \frac{\text{intensity}}{0{,}03/1{,}45}\right) \tag{21}$$

Ứng viên hữu thanh bị trừ đi một lượng chi phí quãng tám:

$$\text{octave_cost} = 0{,}01 \cdot \log_2\left(\frac{\text{Pitch ceiling}}{f}\right) \tag{22}$$

Hàm mục tiêu quy hoạch động tối ưu hóa tổng độ mạnh của các ứng viên trừ đi hai chi phí chuyển tiếp: chi phí nhảy quãng tám (`octave-jump cost` = 0,35) và chi phí chuyển đổi hữu thanh/vô thanh (`voiced/unvoiced cost` = 0,14).

Mười tham số chuẩn của Praat và hành vi ở $f_s = 16$ kHz

Lệnh chuẩn trong giao diện Praat là `Sound: To Pitch (raw autocorrelation)...` tiếp nhận đúng **10 tham số** cấu hình:

- **Time step** (mặc định 0,0 s, tự động): Praat tự tính $\Delta t = 0{,}75/f_{\text{min}} = 0{,}75/75 = 10$ ms, tương ứng bước nhảy khung (`hop`) đúng 160 mẫu tại 16 kHz.
- **Pitch floor** (mặc định 75 Hz): Đặt giới hạn tần số thấp nhất, quyết định cửa sổ phân tích dài 3 chu kỳ ($3/75 = 40$ ms $= 638$ mẫu) và độ trễ tối đa quét là $\tau_{\text{max}} = 214$ mẫu.
- **Pitch ceiling** (mặc định 600 Hz): Đặt giới hạn tần số cao nhất, quyết định độ trễ tối thiểu quét là $\tau_{\text{min}} = \lfloor 16000/600 \rfloor = 26$ mẫu; mọi ứng viên có tần số vượt 600 Hz bị coi là vô thanh.
- **Max. number of candidates** (mặc định 15): Số lượng ứng viên cao độ tối đa được giữ lại ở mỗi khung cho bộ giải thuật Viterbi; mã nguồn tự động ép giá trị này tối thiểu bằng $\lfloor 600/75 \rfloor = 8$.
- **Very accurate** (mặc định `off`): Khi tắt, thuật toán dùng cửa sổ Hanning 3 chu kỳ và nội suy sinc 70 điểm; nếu bật, Praat chuyển sang cửa sổ Gaussian 6 chu kỳ và nội suy sinc 700 điểm.
- **Silence threshold** (mặc định 0,03): Ngưỡng tỉ số biên độ localPeak/globalPeak, kích hoạt vùng dốc phi tuyến nhằm hạ độ mạnh của ứng viên có thanh trong các đoạn thì thầm hoặc im lặng.
- **Voicing threshold** (mặc định 0,45): Ngưỡng sức mạnh tương quan để ứng viên có thanh được chấp nhận cạnh tranh với ứng viên vô thanh.
- **Octave cost** (mặc định 0,01): Trọng số phạt theo logarit tần số, ưu tiên nhẹ cho các tần số cao hơn để chống lại xu hướng rơi vào bẫy chia đôi quãng tám.
- **Octave-jump cost** (mặc định 0,35): Hệ số phạt khi đường cao độ nhảy vọt đột ngột quá một quãng tám giữa hai khung liên tiếp trong thuật toán Viterbi.
- **Voiced/unvoiced cost** (mặc định 0,14): Hệ số phạt khi có sự chuyển trạng thái giữa có thanh và vô thanh giữa hai khung kề nhau trong thuật toán Viterbi.

Lưu ý về bước lọc mềm mép Nyquist (Soft Upsampling): Trong bài báo gốc của Paul Boersma năm 1993, tác giả có đề xuất một bước tiền xử lý gọi là "soft upsampling": làm mềm phổ biên độ dốc tuyến tính về 0 từ 95% đến 100% tần số Nyquist (tức từ 7,6 kHz đến 8,0 kHz ở $f_s = 16$ kHz) bằng biến đổi FFT nhằm giảm hiện tượng gợn sóng bậc cao trong hàm tương quan. Tuy nhiên, khi kiểm tra kỹ lưỡng toàn bộ mã nguồn của Praat 6.4.12 và Praat 7.0.03, **bước xử lý này hoàn toàn không tồn tại trong đường thực thi `raw autocorrelation`**. Tài liệu hướng dẫn sử dụng hiện hành của Praat cũng đã loại bỏ bước này. Do đó, trong thiết kế phần cứng ở Chương 3, ta bố trí một khối lọc FIR 31-tap tùy chọn nhưng đặt công tắc mặc định là **bypass** khi tiến hành đo đạc đối sánh với Praat để tránh tự tạo ra sai lệch không đáng có.

Bảng đối chiếu thiết kế: Praat phần mềm so với FPGA phần cứng

Bảng 2 tổng hợp toàn diện các quyết định kiến trúc: khâu nào ta bám sát Praat 100%, khâu nào ta chủ động thay thế bằng giải thuật thân thiện với phần cứng, lý do kỹ thuật và tác động định lượng đến phép so sánh.

Bảng 2: Đối chiếu toàn diện giữa thuật toán Praat 7.0.03 và kiến trúc phần cứng FPGA trên DE10-Standard

Khâu xử lý	**Praat 7.0.03 (SW)**	**Thiết kế FPGA (HW)**	**Lý do kỹ thuật phần cứng**	**Ảnh hưởng tới phép so sánh**
1. Cửa sổ và lưới thời gian	$N = 638$ mẫu, hop 160, căn đối xứng cả tệp	Bám đúng $N = 638$, hop 160, nhận lưới thời gian từ HPS	Cùng lưới là điều kiện tiên quyết bắt buộc để so sánh từng khung	Sai lệch thời gian bằng 0; khung phân tích trùng khít 100%.
2. Trung bình cục bộ	Lấy trên 426 mẫu ($106 \ldots 531$), số thực double	426 mẫu, nhân hằng số nghịch đảo $M = 10.082.083$, dịch 32 bit	Tránh bộ chia số nguyên phức tạp, vừa vặn trong bộ nhân cứng DSP 25×24	Trừ trung bình khớp bit 100% với mô hình số nguyên.
3. Tính tự tương quan	FFT 1024 điểm, số thực double 64-bit	Nhân-cộng MAC trực tiếp, toán hạng 17-bit, tích lũy 48-bit	1 khối DSP 18×18 chạy nối tiếp 98.543 nhịp, tiết kiệm hàng nghìn ALM so với lõi FFT	Khớp về mặt toán học; sai số lượng tử fixed-point cực nhỏ ($< 10^{-4}$ cent).
4. Chi phí quãng tám (`octave cost`)	Trừ $0{,}01 \log_2(\text{ceiling}/f)$ vào độ mạnh ứng viên	Chuẩn hóa $R[\tau]$ về 27 bit, trừ số hạng tra bảng $K[\tau]$	Thay thế phép chia và hàm logarit phức tạp bằng phép trừ số nguyên	Khớp hoàn toàn với công thức Praat, không làm méo dạng đỉnh parabol.
5. Tinh chỉnh đỉnh cực đại	Nội suy Parabol sơ bộ rồi tinh chỉnh sinc 70 điểm + Brent	Nội suy Parabol 3 điểm, gộp hai phép chia thành một phép chia duy nhất	Phép nội suy sinc70 lặp Brent quá cồng kềnh cho FPGA; parabol chỉ tốn 1 bộ chia	Lệch $< 1{,}1$ cent trên tín hiệu sạch; lệch tới $\sim$ 33 cent trên tín hiệu ồn 5 dB.
6. Xếp hạng ứng viên	Lưu 15 ứng viên cục bộ, tính độ mạnh sinc cho từng đỉnh	Chỉ lưu 1 ứng viên có điểm số S_c cao nhất trên lưới độ trễ τ	Tiết kiệm thanh ghi và logic quản lý bảng ứng viên phức tạp	Ở tín hiệu ồn có hai đỉnh gần hòa, FPGA có thể chọn đỉnh khác Praat (3/207 khung).
7. Lựa chọn cao độ toàn cục	Quy hoạch động Viterbi (Path Finder) duyệt toàn bộ tệp âm thanh	Bộ lọc trung vị 3 điểm (median-3) xử lý trôi (streaming)	Viterbi cần bộ nhớ lưu trữ toàn bộ tệp và giải thuật đệ quy quay lui; median-3 chỉ trễ 1 khung	Median-3 sửa được các đỉnh gai đơn lẻ nhưng không thay thế được Viterbi; phần chênh lệch đo bằng $R_1 - R_2$.

Đọc bảng và định vị mốc tham chiếu. Bảng đối chiếu trên cho thấy một nguyên tắc thiết kế mạch lạc: các khâu biến đổi tín hiệu vật lý cơ bản (lưới khung, độ dài cửa sổ 638 mẫu, trung bình 426 mẫu, chuẩn hóa cửa sổ và chi phí quãng tám dạng trừ) được hiện thực trung thành tuyệt đối với Praat để loại trừ mọi sai số tiên nghiệm.

Ba sự khác biệt có chủ ý nằm ở: (1) nội suy parabol thay cho sinc70 + Brent, (2) chọn đỉnh tốt nhất trên lưới thay vì 15 ứng viên, và (3) bộ lọc trung vị median-3 thay cho Viterbi. Chính vì FPGA không chứa bộ giải thuật Viterbi, trong toàn bộ tài liệu này ta định nghĩa hai mốc tham chiếu Praat chuẩn:

- **Praat** R_1: Chạy Praat với toàn bộ 10 tham số mặc định (gồm cả thuật toán Viterbi với `octave-jump cost` = 0,35 và `V/UV cost` = 0,14).
- **Praat** R_2: Chạy Praat với cấu hình tương tự nhưng đặt `octave-jump cost` = 0 và `V/UV cost` = 0. Khi triệt tiêu hai chi phí chuyển tiếp này, thuật toán Viterbi tự động suy biến thành phép chọn cực đại độc lập trên từng khung (argmax cục bộ).

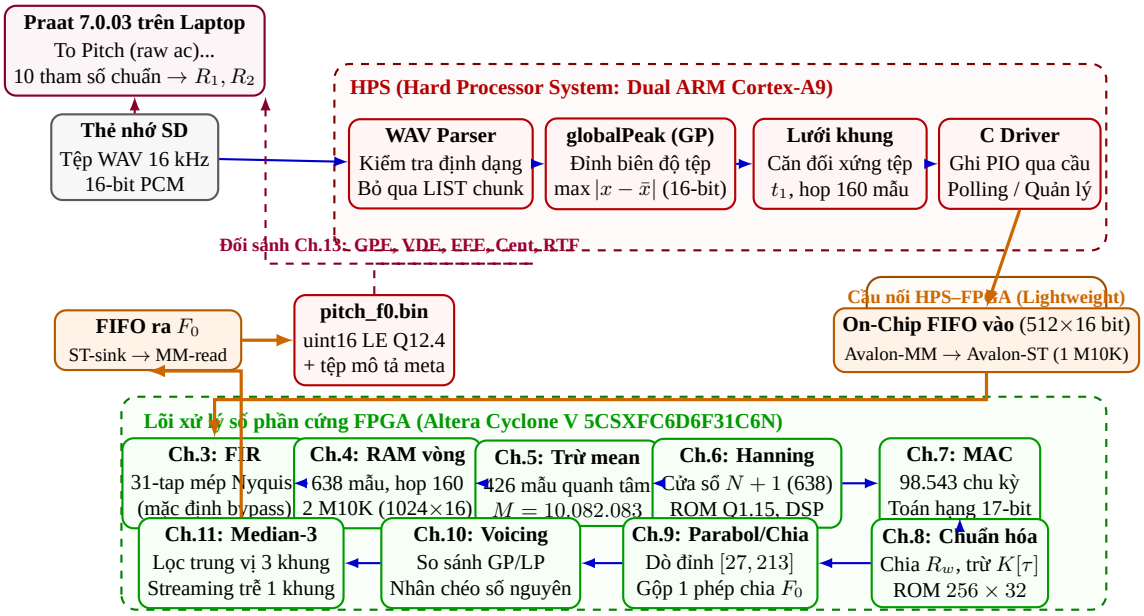
Nhờ định nghĩa này, phép so sánh giữa FPGA (trước bộ lọc median) và Praat R_2 trở thành một phép so sánh **ngang hàng tuyệt đối (apples-to-apples)**: cả hai đều chọn ứng viên tốt nhất trên từng khung độc lập. Hiệu số giữa Praat R_1 và Praat R_2 sẽ bóc tách và định lượng chính xác phần đóng góp riêng biệt của thuật toán Viterbi, giúp anh Tú trả lời trọn vẹn câu hỏi: phần chênh lệch nào do phần cứng số nguyên gây ra, và phần nào do sự vắng mặt của thuật toán quy hoạch động toàn cục.

Tóm lại: Praat vận hành qua 8 bước chặt chẽ với 10 tham số chuẩn; trên FPGA ta bám sát Praat ở kích thước cửa sổ 638 mẫu, trung bình 426 mẫu và chuẩn hóa cửa sổ, đồng thời thay thế sinc70+Brent bằng parabol và Viterbi bằng median-3, sử dụng mốc Praat R_2 làm chuẩn đối sánh ngang hàng trực tiếp.

0.4. Kiến trúc hệ thống và bản đồ các chương

Để hiện thực hóa thuật toán ước lượng cao độ kiểu Praat trên hệ thống phần cứng dị thể SoC (System-on-Chip) Altera Cyclone V (DE10-Standard), toàn bộ quy trình xử lý được phân bổ khoa học giữa hệ thống vi xử lý ARM Cortex-A9 (HPS) và vùng logic khả trình FPGA.

Hình 1 phác họa toàn cảnh kiến trúc luồng dữ liệu hai chiều: đi từ tệp âm thanh WAV trên thẻ nhớ SD, qua phần mềm HPS, đổ vào bộ đệm FIFO thông qua cầu nối bus Lightweight, chạy qua chuỗi xử lý số tín hiệu nối tiếp trong FPGA, rồi quay trở lại HPS để ghi tệp nhị phân kết quả, song song với nhánh đối chứng Praat 7.0.03 chạy trên máy tính cá nhân.



Hình 1: Kiến trúc tổng thể hệ thống ước lượng cao độ F_0: luồng dữ liệu từ tệp WAV qua HPS, cầu Lightweight, lõi xử lý phần cứng FPGA, đường hoàn trả kết quả và nhánh đối sánh với Praat trên máy tính

Đọc hình. Hãy quan sát kỹ từng phân vùng chức năng trên sơ đồ kiến trúc:

1. **Phân vùng HPS (màu đỏ):** Bắt đầu từ tệp âm thanh WAV chuẩn nằm trên thẻ nhớ SD. Khối `WAV Parser` đọc và bóc tách cấu trúc nhị phân RIFF/WAVE, kiểm tra tần số lấy mẫu 16 kHz, định dạng 16-bit PCM mono và bỏ qua các khối mở rộng (như khối `LIST`). Khối tiếp theo tính toán đỉnh biên độ toàn cục globalPeak (GP) của cả tệp. Khối `Lưới khung` tính số khung, mốc thời gian t_1 và chỉ số mẫu bắt đầu của từng khung theo công thức đối xứng của Praat. Cuối cùng, khối điều khiển `C Driver` thực hiện ánh xạ bộ nhớ (`mmap`) thanh ghi cầu nối và phát các lệnh ghi mẫu PCM theo cơ chế PIO (Programmed I/O).

2. **Vùng cầu nối và hàng đợi (màu cam):** Cầu nối `Lightweight HPS-to-FPGA` (32-bit Avalon-MM) tiếp nhận các giao dịch ghi từ vi xử lý. Dữ liệu PCM được nạp vào khối bộ nhớ đệm `On-Chip FIFO` (dung lượng 512 từ 16-bit, chiếm đúng 1 khối RAM M10K) có chức năng đệm mẫu và chuyển đổi giao thức từ bus nhớ Avalon-MM sang giao tiếp luồng Avalon-ST (`valid/ready`).

3. **Phân vùng lõi xử lý phần cứng FPGA (màu xanh lá):** Các mẫu âm thanh lần lượt đi qua chuỗi khối chuyên biệt:

   - Khối lọc `FIR 31-tap` làm mềm mép phổ Nyquist (có công tắc bypass khi đo đối chiếu Praat);
   - Bộ đệm `RAM vòng` 1024 mẫu (sử dụng 2 khối M10K) tích lũy đủ 638 mẫu cho mỗi khung và dịch chuyển mốc đọc `hop` = 160 mẫu;
   - Khối `Trừ trung bình` cục bộ trên 426 mẫu quanh tâm khung, sử dụng phép nhân hằng số nghịch đảo $M = 10.082.083$ trên bộ nhân DSP cứng 25×24;
   - Khối `Cửa sổ Hanning` nhân khung tín hiệu với mảng trọng số 638 mẫu lưu trong bộ nhớ ROM Q1.15;
   - Khối tích lũy `MAC tự tương quan` sử dụng 1 khối DSP 18×18 chạy liên tục 98.543 chu kỳ để tính năng lượng $R[0]$ và 189 giá trị $R[\tau]$ ($\tau = 26 \ldots 214$);

- Khối `Chuẩn hóa và Octave` tra bảng ROM 256×32 để nhân hệ số bù $C[\tau]$ và trừ chi phí quãng tám $K[\tau]$;
- Khối `Parabol và Bộ chia` dò tìm đỉnh cục bộ lớn nhất trên dải $[27, 213]$ và thực thi một phép chia số nguyên duy nhất để tính F_0 định dạng Q12.4;
- Khối `Quyết định Voicing` so sánh độ mạnh đỉnh với năng lượng và biên độ GP, LP qua bất đẳng thức nhân chéo số nguyên;
- Khối `Bộ lọc Median-3` nhận chuỗi kết quả theo cơ chế dòng (streaming), loại bỏ các gai cao độ đơn lẻ với độ trễ cố định 1 khung.

4. **Đường trả kết quả và nhánh đối chứng (màu tím):** Giá trị F_0 tính được cùng trạng thái có thanh/vô thanh được đẩy vào FIFO ngõ ra F0. Bộ điều khiển HPS đọc từ FIFO này qua cầu nối và ghi tuần tự vào tệp nhị phân `pitch_f0.bin` kèm tệp mô tả meta. Song song trên máy tính, phần mềm Praat 7.0.03 xử lý cùng tệp WAV với 10 tham số chuẩn, xuất ra hai đường tham chiếu R_1 và R_2 để phục vụ phân tích sai số toàn diện ở Chương 13.

Bản đồ phân công nội dung chi tiết theo từng chương

Mỗi khối chức năng trong kiến trúc tổng thể trên được nghiên cứu, thiết kế và kiểm chứng chi tiết trong từng chương chuyên biệt của tài liệu này:

- **Chương 1 (WAV trên HPS và Cầu giao tiếp):** Phân tích định dạng WAV nhị phân, giải thuật tính GP và lưới khung Praat bằng C; phân tích đánh đổi lưu lượng và thiết kế On-Chip FIFO (IP `altera_avalon_fifo` và HDL tự viết).
- **Chương 2 (Giao thức Avalon-ST và Mã hóa FSM):** Bản chất giao tiếp bắt tay `valid/ready`, quy tắc `readyLatency` 0 và 1, ngân sách chu kỳ chế độ tệp (file-driven), và so sánh tài nguyên các kiểu mã hóa máy trạng thái FSM (One-Hot, Binary, Gray).
- **Chương 3 (Lọc mép Nyquist FIR):** Hiện thực bộ lọc FIR 31-tap đối xứng bằng FIR Compiler II và HDL gấp tap qua pre-adder; phân tích đáp ứng tần số và cơ chế bypass khi so với Praat.
- **Chương 4 (Chia khung và RAM vòng):** Kỹ thuật đệm trượt 638 mẫu trên RAM vòng 1024×16 (2 khối M10K) với chế độ đọc trước ghi (read-during-write) và quản lý con trỏ cơ sở `rd_base`.
- **Chương 5 (Trừ trung bình cục bộ và DC-Blocker):** Thuật toán trừ trung bình 426 mẫu bằng phép nhân nghịch đảo khớp bit $M = 10.082.083$, $k = 32$; bộ lọc IIR DC-blocker tùy chọn $\alpha = 1 - 2^{-8}$.
- **Chương 6 (Cửa sổ Hanning $N + 1$):** Thiết kế bộ nhớ ROM chứa 638 hệ số Hanning định dạng Q1.15, tối ưu hóa một nửa ROM đối xứng và bộ nhân có chốt thanh ghi.
- **Chương 7 (Tự tương quan bằng MAC):** Trục tính toán nặng nhất của hệ thống: dẫn xuất 98.540 phép nhân-cộng 17-bit, cấu hình khối DSP cứng 18×18 kết hợp cờ `sload` để chuyển độ trễ với 0 nhịp nghỉ.
- **Chương 8 (Chuẩn hóa cửa sổ và Chi phí quãng tám):** Bộ nhớ ROM 256×32 chứa hệ số $C[\tau]$ và $K[\tau]$, kỹ thuật chuẩn hóa $R[\tau]$ về 27 bit để phép nhân vừa vặn trong 1 khối DSP.
- **Chương 9 (Dò đỉnh Parabol và Bộ chia gộp):** Điều kiện tìm đỉnh cục bộ trên dải $[27, 213]$, công thức nội suy parabol đúng dấu, và kiến trúc bộ chia số nguyên LPM_DIVIDE gộp cả nội suy lẫn đổi ra Hz thành một phép chia duy nhất.
- **Chương 10 (Quyết định Voicing):** Bất đẳng thức nhân chéo số nguyên kiểm tra năng lượng và biên độ GP, LP, thay thế hoàn toàn phép chia số thực của Praat.
- **Chương 11 (Bộ lọc trung vị Median-3):** Mạch so sánh lọc trung vị 3 khung nối tiếp, đánh giá khả năng sửa lỗi nhảy quãng tám và tác động tới sai số phân loại vô thanh.
- **Chương 12 (Tích hợp hệ thống và FSM đỉnh):** Kết nối liên hoàn toàn bộ các khối phần cứng, lập bảng phân bổ chu kỳ từng khung, bộ đếm chu kỳ hiệu năng và đóng gói tệp `pitch_f0.bin`.
- **Chương 13 (Kiểm chứng và Đo đạc đối sánh):** Thực hiện đo đạc 3 chiều: sai số chất lượng (GPE, VDE, FFE, cent) so với Praat R_1, R_2; đo thời gian ba phía (chu kỳ FPGA, HPS end-to-end, Praat trên máy tính); bảng so sánh tài nguyên IP và HDL từ báo cáo Fitter.

- **Chương 14 (Bảng kiểm tra lỗi Checklist):** Bảng rà soát 25 tiêu chí kỹ thuật: độ rộng bit, tràn số, độ trễ nhịp, tham số IP và bảo đảm tính tái lập của mã nguồn.
- **Phụ lục A (Bộ chuyển đổi tần số lấy mẫu Resampling):** Thiết kế khối lọc đa pha chuyển đổi tần số âm thanh từ 48 kHz và 44,1 kHz về chuẩn 16 kHz.

Tóm lại: Hệ thống phân chia rành mạch giữa HPS (quản lý tệp, căn lưới thời gian, giao tiếp bus) và FPGA (xử lý số tín hiệu nối tiếp theo dòng); từng khối phần cứng được ánh xạ tương ứng vào từng chương độc lập với kiểm chứng song song hai đường IP và HDL tự viết.

0.5. Các phép đo đánh giá: GPE, VDE, FFE và sai số cent

Để so sánh đường cao độ do mạch số FPGA ước lượng với đường cao độ tham chiếu chuẩn của phần mềm Praat (hoặc cao độ thực *ground-truth* của tệp thử nghiệm), ta không thể dùng các độ đo sai số trung bình bình phương (MSE) thông thường. Tiếng nói gồm các đoạn có thanh (voiced, dây thanh rung) xen kẽ các đoạn vô thanh (unvoiced, âm ma sát hoặc khoảng lặng). Khi một khung bị đoán sai trạng thái thanh tính, giá trị tần số hoàn toàn không xác định, khiến phép trừ số học trực tiếp mất ý nghĩa.

Dẫn xuất các tình huống xảy ra trên một khung phân tích

Xét một khung phân tích bất kỳ tại mốc thời gian t. Gọi F_{ref} là tần số cơ bản của Praat (hoặc ground-truth) và F_{test} là tần số do FPGA tính toán (quy ước quy đổi giá trị 0 đại diện cho trạng thái vô thanh). So sánh giữa hai giá trị này chỉ có thể rơi vào đúng **bốn tình huống vật lý** sau:

- **Tình huống 1: Cả hai bên cùng hữu thanh và gần nhau ($F_{\text{ref}} > 0, F_{\text{test}} > 0$, sai số tương đối $\leq 20\%$).** Đây là các khung ước lượng thành công về mặt cao độ cơ bản. Phần chênh lệch nhỏ giữa hai bên phản ánh sai số làm tròn số học, sự khác biệt giữa thuật toán nội suy đỉnh parabol và nội suy hàm sinc. Tình huống này được đưa vào tập đo **sai số cent mịn (fine-cent error)**.
- **Tình huống 2: Cả hai bên cùng hữu thanh nhưng lệch nhau rất xa ($F_{\text{ref}} > 0, F_{\text{test}} > 0$, sai số tương đối $> 20\%$).** Đây là các lỗi thô nghiêm trọng về mặt giải thuật, thường do mạch số chọn nhầm hài âm bậc hai (nhân đôi quãng tám: $F_{\text{test}} \approx 2F_{\text{ref}}$) hoặc chọn nhầm độ trễ gấp đôi (chia đôi quãng tám: $F_{\text{test}} \approx 0{,}5F_{\text{ref}}$). Hiện tượng này được định nghĩa là **Lỗi cao độ thô (Gross Pitch Error – GPE)**.
- **Tình huống 3: Quyết định thanh tính không đồng nhất (một bên có thanh, bên kia vô thanh).** Hoặc Praat phát hiện có thanh mà FPGA bỏ sót ($F_{\text{ref}} > 0, F_{\text{test}} = 0$), hoặc ngược lại FPGA nhận diện nhầm tiếng ồn thành có thanh ($F_{\text{ref}} = 0, F_{\text{test}} > 0$). Hiện tượng này được định nghĩa là **Lỗi quyết định thanh tính (Voicing Decision Error – VDE)**.
- **Tình huống 4: Cả hai bên cùng vô thanh ($F_{\text{ref}} = 0, F_{\text{test}} = 0$).** Hai bên hoàn toàn đồng thuận về khoảng lặng hoặc âm vô thanh; khung phân tích hoàn toàn chính xác và không có lỗi nào phát sinh.

Từ bốn tình huống trên, cộng đồng xử lý tiếng nói quốc tế chuẩn hóa bốn chỉ số định lượng:

1. Tỉ lệ lỗi quyết định thanh tính (VDE – Voicing Decision Error): Là tỉ số giữa số khung có quyết định thanh tính khác nhau (N_{VDE}, tức Tình huống 3) trên tổng số khung phân tích N:

$$\text{VDE} = \frac{N_{\text{VDE}}}{N} = \frac{\sum_{i=1}^{N} \left[(F_{\text{ref},i} > 0) \oplus (F_{\text{test},i} > 0) \right]}{N} \tag{23}$$

2. Tỉ lệ lỗi cao độ thô (GPE – Gross Pitch Error): Chỉ tính riêng trên tập hợp các khung mà *cả hai bên đều nhận định là có thanh* (N_{both} khung). GPE là tỉ lệ số khung bị lệch tần số quá 20% (N_{GPE}, tức Tình huống 2) trên N_{both}:

$$\text{GPE} = \frac{N_{\text{GPE}}}{N_{\text{both}}} = \frac{\sum_{i \in \text{both}} \mathbb{I}\left(\left| \frac{F_{\text{test},i}}{F_{\text{ref},i}} - 1 \right| > 0{,}20 \right)}{N_{\text{both}}} \tag{24}$$

3. Tỉ lệ lỗi tổng hợp toàn khung (FFE – F_0 Frame Error): Phản ánh bức tranh toàn cảnh về độ chính xác của hệ thống, bằng tổng số khung bị lỗi thanh tính cộng số khung bị lỗi cao độ thô, chia cho tổng số khung N:

$$\text{FFE} = \frac{N_{\text{VDE}} + N_{\text{GPE}}}{N} \tag{25}$$

4. Thống kê sai số cent mịn (Fine-Cent Statistics): Được tính riêng biệt trên các khung ở Tình huống 1 (cùng có thanh và không phạm lỗi thô GPE). Độ lệch cao độ tính bằng cent:

$$C_i = 1.200 \cdot \log_2 \left(\frac{F_{\text{test},i}}{F_{\text{ref},i}} \right) \tag{26}$$

Ta báo cáo bốn đại lượng thống kê: trung bình độ lệch tuyệt đối (Mean $|C|$), độ lệch chuẩn (Std C), phân vị 95 (P95 $|C|$) và sai số mịn cực đại (max $|C|$).

Ý nghĩa của ngưỡng 20% **và vì sao phải loại bỏ lỗi thô khi đo cent?** Một ngưỡng lệch tương đối 20% về mặt tần số tương ứng với bao nhiêu trong cảm thụ âm nhạc? Ta quy đổi trực tiếp ra cent:

$$1.200 \cdot \log_2(1{,}20) \approx 315{,}64 \text{ cent} \approx 3{,}16 \text{ nửa cung (semitones)} \tag{27}$$

Khoảng cách hơn 3 nửa cung vượt xa ngưỡng phân biệt cao độ mịn của con người (thường dưới 20–50 cent). Khi một khung bị lỗi nhảy quãng tám (nhân đôi tần số: $F_{\text{test}} = 2F_{\text{ref}}$), sai số cent lên tới đúng 1.200 cent! Nếu gộp các khung lỗi quãng tám này vào thống kê cent mịn, chỉ cần một khung lỗi 1.200 cent duy nhất cũng đủ làm biến dạng giá trị trung bình từ $0{,}2$ cent vọt lên hàng chục cent, che lấp hoàn toàn độ chính xác vi mô của bộ nội suy đỉnh parabol. Do đó, việc tách rời GPE (đánh giá lỗi giải thuật) và cent mịn (đánh giá độ tinh xảo số học) là nguyên tắc bắt buộc.

Ví dụ tính tay thu nhỏ trên 10 khung mô phỏng

Để người đọc tự tay kiểm tra từng con số, xét kịch bản mẫu gồm 10 khung phân tích đại diện đầy đủ cho 4 tình huống vật lý (được sinh và tính toán bởi kịch bản `verify_v3/ch00_metrics.py`):

Bảng 3: Bảng tính tay chi tiết các chỉ số chất lượng trên 10 khung mô phỏng sư phạm

Khung	F_{ref} (Hz)	F_{test} (Hz)	Phân loại tình huống	Sai số tương đối	Sai số cent	Trạng thái lỗi
0	0,0	0,0	Cả hai vô thanh (TH 4)	—	—	Không lỗi
1	120,0	120,0	Cùng có thanh (TH 1)	$0{,}00\%$	$+0{,}00$	Khớp tuyệt đối
2	150,0	152,0	Cùng có thanh (TH 1)	$1{,}33\%$	$+22{,}93$	Sai số mịn
3	200,0	196,0	Cùng có thanh (TH 1)	$2{,}00\%$	$-34{,}98$	Sai số mịn
4	220,0	224,0	Cùng có thanh (TH 1)	$1{,}82\%$	$+31{,}19$	Sai số mịn
5	0,0	130,0	Lệch thanh tính (TH 3)	—	—	**Lỗi VDE**
6	140,0	0,0	Lệch thanh tính (TH 3)	—	—	**Lỗi VDE**
7	100,0	200,0	Nhân đôi quãng tám (TH 2)	$100{,}00\%$	$+1.200{,}00$	**Lỗi GPE**
8	300,0	150,0	Chia đôi quãng tám (TH 2)	$50{,}00\%$	$-1.200{,}00$	**Lỗi GPE**
9	0,0	0,0	Cả hai vô thanh (TH 4)	—	—	Không lỗi

Dẫn xuất từng chỉ số từ Bảng 3:

- Tổng số khung: $N = 10$.
- Số khung cùng có thanh: $N_{\text{both}} = 6$ (Khung 1, 2, 3, 4, 7, 8).
- Lỗi thanh tính (VDE): xuất hiện ở Khung 5 và Khung 6 ($N_{\text{VDE}} = 2$). Tỉ lệ:

$$\text{VDE} = \frac{2}{10} = 20{,}0\% \tag{28}$$

- Lỗi cao độ thô (GPE): trong 6 khung cùng có thanh, Khung 7 bị nhân đôi cao độ ($100\% > 20\%$) và Khung 8 bị chia đôi ($50\% > 20\%$), nên $N_{\text{GPE}} = 2$. Tỉ lệ:

$$\text{GPE} = \frac{2}{6} \approx 33{,}3\% \tag{29}$$

- Lỗi tổng hợp toàn khung (FFE):

$$\text{FFE} = \frac{N_{\text{VDE}} + N_{\text{GPE}}}{N} = \frac{2+2}{10} = 40{,}0\% \tag{30}$$

- Thống kê sai số cent mịn trên 4 khung không lỗi thô (Khung 1, 2, 3, 4 với độ lệch tuyệt đối là $0{,}00$, $22{,}96$, $34{,}97$, $31{,}18$ cent):

$$\text{Trung bình tuyệt đối} = \frac{0{,}00 + 22{,}96 + 34{,}97 + 31{,}18}{4} = 22{,}28 \text{ cent} \tag{31}$$

$$\text{Độ lệch chuẩn (Std)} = 25{,}64 \text{ cent} \tag{32}$$

$$\text{Phân vị 95 (P95)} = 34{,}41 \text{ cent} \tag{33}$$

$$\text{Sai số mịn cực đại} = 34{,}98 \text{ cent} \tag{34}$$

Tất cả các hàm đo đạc trong `verify_v3/ch00_metrics.py` đã được đối chiếu và khẳng định cho kết quả trùng khớp hoàn toàn với kịch bản tổng thể `golden_v2/compare.py` khi kiểm tra trên hai tệp kiểm chứng `synth_glide_vuv_16k.wav` và `synth_glide_vuv_16k_snr5.wav`.

Tóm lại: VDE đo sự sai lệch phân loại hữu thanh/vô thanh trên toàn tệp, GPE đo lỗi nhảy quãng tám trên các khung cùng hữu thanh, FFE gộp cả hai loại lỗi trên tổng số khung, còn sai số cent mịn đánh giá độ chính xác toán học của bộ nội suy đỉnh sau khi đã bóc tách triệt để các lỗi thô.

0.6. Ngân sách chu kỳ, hệ số thời gian thực RTF, và mạch phát cặp mẫu phần cứng

Khi chuyển dịch một thuật toán từ môi trường máy tính sang chip bán dẫn FPGA, câu hỏi sống còn đầu tiên của người kỹ sư là: mạch số có đủ nhanh để xử lý tín hiệu theo thời gian thực (real-time) hay không, và ta cần bao nhiêu tài nguyên phần cứng (bộ nhân DSP, khối nhớ RAM M10K) để đáp ứng thời hạn đó?

Dẫn xuất ngân sách chu kỳ từ nguyên lý đầu tiên

Hệ thống phần cứng trên bo mạch DE10-Standard vận hành với xung nhịp đồng hồ hệ thống $f_{\text{clk}} = 50$ MHz (chu kỳ xung nhịp $T_{\text{clk}} = 20$ ns). Tín hiệu âm thanh chuẩn được lấy mẫu ở tần số $f_s = 16.000$ Hz (chu kỳ lấy mẫu $T_s = 62{,}5$ μs).

Số chu kỳ xung nhịp đồng hồ FPGA có sẵn giữa hai mẫu âm thanh liên tiếp là:

$$N_{\text{cycles/sample}} = \frac{f_{\text{clk}}}{f_s} = \frac{50.000.000}{16.000} = 3.125 \text{ chu kỳ/mẫu} \tag{35}$$

Cứ sau một khoảng thời gian bước nhảy khung $\Delta t = 10$ ms (`hop` = 160 mẫu), hệ thống phải hoàn tất việc tính toán một giá trị F_0 mới. Ngân sách chu kỳ tối đa cho mỗi khung để đạt ngưỡng thời gian thực là:

$$N_{\text{budget/hop}} = 160 \text{ mẫu} \times 3.125 \text{ chu kỳ/mẫu} = 500.000 \text{ chu kỳ/khung} \tag{36}$$

Con số 500.000 chu kỳ ở 50 MHz chính là trần thời gian thực. Bất kỳ thiết kế nào hoàn thành một khung dưới 500.000 chu kỳ đều chạy nhanh hơn tốc độ phát âm thanh thực tế.

Dẫn xuất số phép nhân-cộng MAC và số chu kỳ của khối tự tương quan

Khối tiêu tốn chu kỳ tính toán nặng nề nhất trong toàn bộ hệ thống là khối tự tương quan. Ta xét một khung có độ dài $N = 638$ mẫu, rà soát trên dải độ trễ từ $\tau_{\text{min}} = 26$ đến $\tau_{\text{max}} = 214$ (tương ứng với $214 - 26 + 1 = 189$ độ trễ).

Tại một độ trễ τ bất kỳ, số cặp mẫu cần nhân và tích lũy là $N - \tau = 638 - \tau$. Tổng số phép nhân-cộng (MAC) cho toàn bộ 189 độ trễ được tính chính xác bằng công thức tổng cấp số cộng:

$$\sum_{\tau=26}^{214} (638 - \tau) = \sum_{\tau=26}^{214} 638 - \sum_{\tau=26}^{214} \tau \tag{37}$$

$$= 189 \times 638 - \frac{189 \times (26 + 214)}{2} \tag{38}$$

$$= 120.582 - \frac{189 \times 240}{2} = 120.582 - 189 \times 120 \tag{39}$$

$$= 120.582 - 22.680 = 97.902 \text{ phép MAC} \tag{40}$$

Để tính năng lượng khung chuẩn hóa tại gốc $\tau = 0$ ($R[0]$), ta cần thêm $N = 638$ phép nhân-cộng nữa:

$$N_{\text{pairs}} = 97.902 + 638 = 98.540 \text{ cặp mẫu} \tag{41}$$

Khối nhân-cộng phần cứng trong Chương 7 được thiết kế với đường ống 2 tầng thanh ghi (thanh ghi ngõ vào toán hạng trong khối DSP cứng và thanh ghi tổng tích lũy ngõ ra), cộng với 1 nhịp đọc từ bộ nhớ RAM M10K hai cổng, tạo ra độ trễ đường ống (pipeline latency) tổng cộng là 3 chu kỳ xung nhịp.

Nhờ cơ chế phát cặp mẫu liên tục không nhịp nghỉ (sẽ phân tích ở phần HDL bên dưới), tổng số chu kỳ xung nhịp từ lúc phát cặp địa chỉ đầu tiên đến khi giá trị $R[214]$ xuất hiện hợp lệ ở đầu ra là:

$$N_{\text{cycles, MAC}} = 98.540 \text{ nhịp phát} + 3 \text{ nhịp trễ} = 98.543 \text{ chu kỳ/khung} \tag{42}$$

Con số 98.543 chu kỳ này đã được đo đạc chính xác tuyệt đối bằng bộ mô phỏng iverilog trên mã nguồn `golden_v2/hdl_check/tb_mac638.v`.

Khái niệm Hệ số thời gian thực (RTF) và Độ trễ so với Thông lượng

Hệ số thời gian thực (**RTF – Real-Time Factor**) được định nghĩa là tỉ số giữa thời gian xử lý của phần cứng chia cho thời lượng thực tế của đoạn âm thanh:

$$\text{RTF} = \frac{T_{\text{xử lý}}}{T_{\text{âm thanh}}} = \frac{\text{Số chu kỳ FPGA}}{N_{\text{budget/hop}}} \tag{43}$$

Riêng đối với khối tự tương quan MAC, hệ số RTF đạt được là:

$$\text{RTF}_{\text{MAC}} = \frac{98.543}{500.000} \approx 0{,}197086 \approx 0{,}197 \tag{44}$$

Tổng số chu kỳ của toàn bộ lõi FPGA (bao gồm cả trừ trung bình 426 mẫu, nhân cửa sổ Hanning 638 mẫu, chuẩn hóa cửa sổ, nội suy parabol và lọc median-3) ước lượng chỉ mất khoảng 99.700 chu kỳ/khung (sẽ phân tích chi tiết ở Chương 12):

$$\text{RTF}_{\text{core}} = \frac{99.700}{500.000} \approx 0{,}1994 \approx 0{,}199 \tag{45}$$

Hệ số RTF $\approx 0{,}20$ có nghĩa là: mạch số FPGA chỉ mất 2 mili-giây để tính toán xong một khung âm thanh dài 10 mili-giây! Phần cứng chạy nhanh gấp 5 lần tốc độ phát âm thanh thời gian thực.

Chỗ dễ vướng: Độ trễ đường ống (Latency) có làm chậm 98.540 phép tính không? Câu trả lời là hoàn toàn không. Người học cần phân biệt rành mạch giữa *độ trễ (latency)* và *thông lượng (throughput)*. Hãy liên tưởng đến một băng chuyền nhà máy: món hàng đầu tiên mất 3 nhịp di chuyển mới đến điểm cuối của băng chuyền (latency = 3 chu kỳ). Nhưng kể từ thời điểm đó trở đi, cứ mỗi nhịp xung nhịp đồng hồ lại có một món hàng hoàn thiện lăn ra khỏi băng chuyền (throughput = 1 kết quả/nhịp). Vì 98.540 cặp mẫu được phát liên tiếp không nghỉ, ta thu được 98.540 kết quả chỉ sau $98.540 + 3 = 98.543$ chu kỳ. Khoảng trễ 3 chu kỳ trên tổng số gần một trăm nghìn chu kỳ chỉ chiếm tỉ lệ vỏn vẹn $3/98.543 \approx 0{,}003\%$, hoàn toàn không đáng kể.

Ba phép đo thời gian và định nghĩa RTF trong hệ thống

Trong báo cáo kiểm thử toàn tuyến (Chương 13), ta đo đạc và so sánh ba mốc thời gian độc lập:

1. **Thời gian lõi FPGA (Core Cycle Count):** Đo bằng thanh ghi đếm chu kỳ phần cứng (`cycle counter`) tích hợp trực tiếp trên FPGA, bấm giờ từ lúc cờ `start` bật đến khi cờ `done` tích cực; quy đổi $\text{RTF}_{\text{FPGA}} = \text{Số chu kỳ}/500.000$.
2. **Thời gian toàn trình HPS (End-to-End HPS Time):** Đo bằng hàm chuẩn POSIX `clock_gettime` (với nguồn xung `CLOCK_MONOTONIC`) trên hệ điều hành Linux của HPS, bao gồm toàn bộ thời gian: đọc tệp WAV từ thẻ SD, bóc tách cấu trúc, tính GP, ghi mẫu qua cầu nối bus, đợi FPGA xử lý, đọc kết quả F0 từ FIFO và ghi tệp nhị phân xuống đĩa.
3. **Thời gian Praat trên máy tính (Laptop Praat Time):** Đo trực tiếp bên trong kịch bản Praat (tệp `praat_pitch.praat`) bằng các lệnh bấm giờ nội bộ, nhằm loại trừ thời gian khởi tạo giao diện GUI và nạp thư viện hệ điều hành. Do Praat mặc định kích hoạt tính toán đa luồng (cơ chế `MelderThread`), số luồng CPU của máy tính xách tay phải được ghi nhận rõ ràng trong báo cáo đo đạc.

Mạch phần cứng Chương 0: Bộ phát cặp mẫu tự tương quan (`ch00_pair_counter`)

Để gắn kết chặt chẽ ngân sách chu kỳ lý thuyết với phần cứng thực tế, ta thiết kế một khối phần cứng tổng hợp được: `rtl/v3/ch00_pair_counter.v`. Khối này có nhiệm vụ phát tuần tự chuỗi địa chỉ $(n, n+\tau)$ để truy xuất bộ nhớ RAM hai cổng, đồng thời sinh ra các cờ điều khiển `sload` (báo nạp tích mới) và `last` (báo kết thúc lag) đúng nhịp cho khối MAC ở Chương 7.

Vì sao không có Intel IP cho khối này? Trong thư viện IP Catalog của Intel Quartus Prime, Intel cung cấp các bộ đếm chuẩn (LPM_COUNTER) hoặc bộ tạo địa chỉ DMA tuyến tính. Tuy nhiên, quy luật duyệt chỉ số tự tương quan $(n, n+\tau)$ trượt hai chiều với giới hạn $n \in [0, N-1-\tau]$ biến thiên theo từng độ trễ τ là một hàm chỉ số phi tuyến tính đặc thù. Không có bất kỳ khối IP nào hỗ trợ sẵn cấu trúc này; việc tự viết mã HDL là lựa chọn tối ưu, linh hoạt và tiết kiệm tài nguyên nhất.

Listing 1: Trích đoạn máy trạng thái và bộ phát cặp mẫu trong `rtl/v3/ch00_pair_counter.v`

```
module ch00_pair_counter #(
    parameter integer N          = 638,
    parameter integer TAU_MIN    = 26,
    parameter integer TAU_MAX    = 214,
    parameter integer ADDR_WIDTH = 10
)(
    input  wire                  clk, rst_n, start,
    input  wire [ADDR_WIDTH-1:0] cfg_n, cfg_tau_min, cfg_tau_max,
    output reg                   busy, done, valid, sload, last,
    output reg  [ADDR_WIDTH-1:0] addr_a, addr_b, tau_out,
    output reg  [31:0]           total_cycles, pair_count
);
    localparam [1:0] STATE_IDLE = 2'b00, STATE_RUN = 2'b01, STATE_DONE = 2'b10;
    reg [1:0]            state;
    reg [ADDR_WIDTH-1:0] n_cnt, tau_cnt, act_n, act_tau_min, act_tau_max;

    wire [ADDR_WIDTH-1:0] max_n = (tau_cnt == 0) ?
                                  (act_n - 1'b1) : (act_n - 1'b1 - tau_cnt);

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            state <= STATE_IDLE;
            busy <= 0; done <= 0; valid <= 0; sload <= 0; last <= 0;
            addr_a <= 0; addr_b <= 0; tau_out <= 0;
            total_cycles <= 0; pair_count <= 0;
            n_cnt <= 0; tau_cnt <= 0;
            act_n <= N; act_tau_min <= TAU_MIN; act_tau_max <= TAU_MAX;
        end else begin
            case (state)
                STATE_IDLE: begin
                    done <= 0;
                    if (start) begin
                        state <= STATE_RUN; busy <= 1; valid <= 1; sload <= 1;
                        act_n <= (cfg_n != 0) ? cfg_n : N;
                        act_tau_min <= (cfg_n != 0) ? cfg_tau_min : TAU_MIN;
                        act_tau_max <= (cfg_n != 0) ? cfg_tau_max : TAU_MAX;
                        addr_a <= 0; addr_b <= 0; tau_cnt <= 0; n_cnt <= 0;
                        last <= ((cfg_n != 0 ? cfg_n : N) == 1);
                        total_cycles <= 1; pair_count <= 1;
                    end
                end
                STATE_RUN: begin
                    total_cycles <= total_cycles + 1'b1;
                    if (n_cnt < max_n) begin
                        n_cnt <= n_cnt + 1'b1;
                        addr_a <= n_cnt + 1'b1;
                        addr_b <= (tau_cnt == 0) ?
                                  (n_cnt + 1'b1) : (n_cnt + 1'b1 + tau_cnt);
                        sload <= 0;
                        last <= (n_cnt + 1'b1 == max_n);
                        pair_count <= pair_count + 1'b1;
                    end else begin
```

```
                        if (tau_cnt == 0) begin // Xong R[0], sang TAU_MIN
                            tau_cnt <= act_tau_min; tau_out <= act_tau_min;
                            n_cnt <= 0; addr_a <= 0; addr_b <= act_tau_min;
                            sload <= 1;
                            last <= (act_n - 1'b1 - act_tau_min == 0);
                            pair_count <= pair_count + 1'b1;
                        end else if (tau_cnt < act_tau_max) begin // Sang tau ke tiep
                            tau_cnt <= tau_cnt + 1'b1; tau_out <= tau_cnt + 1'b1;
                            n_cnt <= 0; addr_a <= 0; addr_b <= tau_cnt + 1'b1;
                            sload <= 1;
                            last <= (act_n - 1'b1 - (tau_cnt + 1'b1) == 0);
                            pair_count <= pair_count + 1'b1;
                        end else begin // Hoan tat tat ca cac lag
                            valid <= 0; sload <= 0; last <= 0; busy <= 0; done <= 1;
                            state <= STATE_DONE;
                        end
                    end
                end
                STATE_DONE: begin
                    busy <= 0; valid <= 0; sload <= 0; last <= 0; done <= 1;
                    if (!start) state <= STATE_IDLE;
                end
            endcase
        end
    end
endmodule
```

Giải thích mã nguồn Verilog từng khối:

- Dòng 1–12 khai báo cổng: nhận các tham số động `cfg_n, cfg_tau_min, cfg_tau_max` (nếu bằng 0 thì dùng tham số tĩnh $N = 638, \tau \in [26, 214]$). Xuất tín hiệu địa chỉ `addr_a, addr_b`, cờ `valid`, `sload`, `last` và các thanh ghi đếm chu kỳ `total_cycles`, `pair_count`.
- Dòng 17 định nghĩa tín hiệu tổ hợp `max_n`: khi $\tau = 0$, mẫu cuối cùng là $N - 1$; khi $\tau \geq \tau_{\min}$, mẫu cuối cùng là $N - 1 - \tau$.
- Dòng 26–37 (`STATE_IDLE`): khi nhận xung kích hoạt `start`, FSM nạp tham số làm việc, khởi tạo địa chỉ $(0, 0)$ cho $R[0]$, bật cờ `valid` = 1 và cờ nạp `sload` = 1.
- Dòng 39–61 (`STATE_RUN`): mạch tăng dần con đếm mẫu n ở mỗi nhịp xung nhịp. Khi n đạt tới `max_n`, mạch kiểm tra chuyển đổi độ trễ: nếu vừa xong $\tau = 0$, mạch nhảy tức thì sang $\tau = \tau_{\min}$ và phát địa chỉ $(0, \tau_{\min})$; nếu đang ở $\tau < \tau_{\max}$, mạch nhảy sang $\tau + 1$ và phát $(0, \tau + 1)$. Tại đúng chu kỳ đầu tiên của mỗi độ trễ, cờ `sload` được kích hoạt mức cao, báo cho khối nhân-cộng nạp tích mới mà không cần bất kỳ nhịp xóa thanh ghi tích lũy nào.
- Dòng 62–66: khi hoàn thành mẫu cuối cùng của độ trễ cực đại $\tau_{\max}$, mạch hạ cờ `valid`, bật cờ hoàn tất `done` và chuyển sang `STATE_DONE`.

Mô hình tương đương Python (Twin) và Kết quả kiểm chứng iverilog: Bộ mô hình tương đương số nguyên viết bằng Python (`verify_v3/ch00_pair_counter_twin.py`) mô phỏng chính xác hành vi phát cặp mẫu và tính giá trị băm kiểm tra (checksum 32-bit):

Listing 2: Mô hình tương đương `PairCounterTwin` trong `verify_v3/ch00_pair_counter_twin.py`

```
class PairCounterTwin:
    def __init__(self, N, tau_min, tau_max):
        self.N, self.tau_min, self.tau_max = N, tau_min, tau_max

    def generate_pairs(self):
        pairs = []
        # Lag 0: R[0]
        for n in range(self.N):
            pairs.append({
                'a': n, 'b': n, 'tau': 0,
                'sload': int(n == 0), 'last': int(n == self.N - 1)
            })
```

```
        # Lags tau_min .. tau_max
        for tau in range(self.tau_min, self.tau_max + 1):
            max_n = self.N - 1 - tau
            for n in range(max_n + 1):
                pairs.append({
                    'a': n, 'b': n + tau, 'tau': tau,
                    'sload': int(n == 0), 'last': int(n == max_n)
                })
        return pairs
```

Kịch bản kiểm thử `verify_v3/ch00_test_pair_counter.py` kết nối với testbench Verilog `tb/v3/tb_ch00_pair_counter.v`, biên dịch bằng `iverilog -g2005` và thực thi bằng `vvp`. Quá trình kiểm chứng bao gồm:

- Cấu hình chuẩn Praat ($N = 638, \tau \in [26, 214]$): Mạch phát đúng 98.540 cặp mẫu trong 98.540 chu kỳ, sinh đúng 190 xung `sload` và 190 xung `last`. Mã băm kiểm tra chuỗi dữ liệu đạt giá trị `0x26518cf3`, trùng khớp 100% với mô hình Python.
- 200 cấu hình ngẫu nhiên (với $N \in [15, 80]$, τ thay đổi tự do): Tất cả 200 trường hợp đều vượt qua kiểm tra, số cặp mẫu và giá trị băm checksum khớp bit đối bit và chu kỳ đối chu kỳ ($201/201$ ca kiểm thử thành công, 0 lỗi, lưu tại `verify_v3/ch00_pair_counter.log`).

Tóm lại: Ở 50 MHz, ngân sách thời gian thực là 500.000 chu kỳ cho mỗi bước nhảy 10 ms; khối tự tương quan cần 98.543 chu kỳ (hệ số RTF $\approx 0{,}197$), được hiện thực bằng mạch phát cặp mẫu liên tục `ch00_pair_counter` chuyển độ trễ với 0 nhịp nghỉ, đã kiểm chứng khớp tuyệt đối 201/201 ca thử nghiệm.

0.1 Bản đồ kiểm chứng ba tầng và bộ golden v2

Vấn đề thực tế: Khi hiện thực một chuỗi xử lý tín hiệu số phức tạp lên phần cứng FPGA (DE10-Standard Cyclone V), nếu ta đưa toàn bộ kiến trúc từ xử lý mẫu đến chia nội suy và xuất dữ liệu lên chip, rồi chỉ so sánh kết quả tần số cơ bản F0 cuối cùng với phần mềm Praat chạy trên máy tính, ta sẽ rơi vào bế tắc hoàn toàn khi xuất hiện sai số. Nếu một khung tín hiệu bị lệch tần số hoặc sai phân loại hữu thanh/vô thanh (V/UV), nguyên nhân bắt nguồn từ đâu?

- Do chỉ số cắt khung lệch một mẫu tại tầng HPS?
- Do khối trừ trung bình cục bộ làm tròn sai hoặc tràn số 17-bit?
- Do bảng ROM Hanning lượng tử hóa sai hệ số tỷ lệ?
- Do khối nhân-cộng tự tương quan MAC bị trễ chu kỳ xung nhịp hoặc sai tín hiệu kích hoạt nạp?
- Hay do phép chia điểm đỉnh parabol số nguyên bị mất mát độ chính xác?

Một sai lệch dù chỉ 1 mẫu hay 1 bit thấp nhất (LSB) ở tầng trước sẽ bị tích lũy và khuếch đại thành sai số hàng trăm cent hoặc sai nhãn V/UV ở tầng sau mà không để lại bất kỳ manh mối nào. Vì vậy, ta cần một kiến trúc kiểm chứng đa tầng chặt chẽ, phân định rạch ròi giữa *tính đúng đắn logic của mạch phần cứng* và *sai số xấp xỉ thuật toán so với Praat*.

0.1.1 Kiến trúc kiểm chứng ba tầng (Three-tier Verification)

Chiến lược kiểm chứng của cuốn sách được xây dựng thành ba tầng độc lập, tạo thành ”tấm lưới bảo vệ” khép kín xuyên suốt từ mã nguồn thuật toán đến chân chip FPGA (Hình 1):

1. **Tầng 1 — Khớp bit tuyệt đối giữa mô hình Python số nguyên và mô phỏng RTL (Twin vs RTL):** Tầng này so sánh đối ứng bit-exact (khớp chính xác 100% từng bit dữ liệu và từng chu kỳ xung nhịp) giữa mô hình tương đương Python (`golden_v2/fixed_twin.py` và các script twin cục bộ của từng chương) với mô phỏng Verilog RTL (`rtl/v3/` và `tb/v3/`) chạy bằng `iverilog/vvp`. Mỗi khối phần cứng phải vượt qua ít nhất 200 véc-tơ kiểm tra ngẫu nhiên cùng các khung tín hiệu biên dồn nén (stress vectors, biên độ cực trị ± 32.768). Tiêu chuẩn nghiệm thu ở tầng 1 là 0 lỗi: chuỗi bit đầu ra và mã băm kiểm tra (checksum) của RTL phải trùng khớp hoàn toàn với mô hình Python. Tầng này loại bỏ triệt để mọi lỗi logic, lỗi tràn số và lỗi sai lệch chu kỳ giao tiếp Avalon-ST.

2. **Tầng 2 — Khớp bit tuyệt đối giữa mô phỏng RTL và phần cứng chạy thực (RTL vs Board):** Tầng này so sánh kết quả mô phỏng RTL với dữ liệu đo thực tế trên kit DE10-Standard, được trích xuất thông qua bộ phân tích logic nhúng Quartus SignalTap II hoặc đọc trực tiếp từ bộ đệm DDR của HPS qua bus Avalon-MM. Tầng 2 nhằm phát hiện các lỗi vật lý nằm ngoài tầm kiểm soát của code RTL thuần túy: khởi tạo bộ nhớ M10K qua file `.mif`/`.hex`, cấu hình bộ chia tần và độ ổn định của khối PLL (50 MHz), ràng buộc chân IO, và sự đồng bộ tín hiệu bắt tay trên bus hệ thống Platform Designer (Qsys). Tiêu chuẩn nghiệm thu: chuỗi kết quả F0 phần cứng trả về HPS phải trùng khít 100% với chuỗi kết quả của mô phỏng RTL.

3. **Tầng 3 — Đánh giá thống kê giữa phần cứng và chuẩn Praat trên máy tính (FPGA/Twin vs Praat):** Sau khi tầng 1 và tầng 2 đảm bảo phần cứng thực thi đúng 100% đặc tả toán học số nguyên, tầng 3 tiến hành so sánh đối chiếu giữa đầu ra của FPGA (hoặc Python Twin) với phần mềm Praat 7.0.03 gốc chạy trên máy tính thông qua công cụ `golden_v2/compare.py`. Thước đo ở tầng này không còn là khớp bit mà là các chỉ số xử lý tín hiệu khách quan: VDE, GPE, FFE và phân bố sai số cent mịn (trung bình, độ lệch chuẩn, phân vị 95, cực đại). Mục tiêu là chứng minh độ lệch âm học giữa giải pháp phần cứng và Praat luôn nằm sâu dưới ngưỡng nghe nhận biết của tai người, đồng thời giải thích tường minh nguyên nhân của từng khung sai lệch.

0.1.2 Cấu trúc bộ công cụ golden v2

Toàn bộ mã nguồn kiểm chứng tham chiếu được đặt tại thư mục `golden_v2/`, đóng vai trò ”trọng tài kỹ thuật” cho toàn bộ tài liệu:

- `constants.py`: Định nghĩa tập trung mọi hằng số suy xuất trực tiếp từ mã nguồn C++ của Praat 7.0.03 (`Sound_to_Pitch.cpp`), bao gồm kiểm tra khẳng định `assert`.
- `frames.py`: Hiện thực hóa chia lưới khung ngắn hạn (`Sampled_shortTermAnalysis`) bằng số thực kép.
- `praat_float_ref.py`: Bản chuyển đổi trực tiếp thuật toán Praat 7.0.03 sang Python số thực chuẩn: trích chọn 15 ứng viên, nội suy sơ bộ parabol, tinh chỉnh độ chính xác cao sinc70 + Brent, bộ dò đường Viterbi (R_1) và đường chi phí bằng 0 (R_2), kết hợp bộ lọc trung vị median-3.
- `fixed_twin.py`: Mô hình số nguyên tương đương bit-exact toàn tuyến. Phần tài liệu docstring của tệp này chính là bản đặc tả số nguyên 10 bước chuẩn hóa mà mạch phần cứng RTL bắt buộc phải tuân theo.
- `gen_roms.py`: Kịch bản tự động sinh bảng ROM Hanning Q1.15 (`rom/hanning638_q15.hex`), ROM bù suy giảm $C[\tau]$ và chi phí $K[\tau]$ (`rom/ck_rom.hex`), cùng tệp `rom/constants.vh`.
- `compare.py`: Kịch bản tự động thực thi chuỗi so sánh đa phương diện, xuất báo cáo đối soát chi tiết (`golden_v2/out/compare_report.md`).

0.1.3 Đặc tả 10 bước số nguyên của lõi phần cứng (fixed_twin.py)

Để mạch phần cứng FPGA có thể hoạt động hoàn toàn bằng số nguyên có dấu với tài nguyên tối thiểu mà không cần khối dấu phẩy động tốn kém, thuật toán ước lượng F0 được phân rã thành 10 bước xử lý số nguyên bit-exact như sau:

1. **Bước 1 — Trừ trung bình cục bộ:** Lấy tổng tích lũy $S = \sum_{j=106}^{531} x[j]$ trên đúng 426 mẫu quanh tâm khung (ứng với hai chu kỳ dài nhất tại floor 75 Hz). Giá trị trung bình nguyên được tính bằng phép nhân-dịch thay thế bộ chia:
$$\text{mean} = \left\lfloor \frac{(S + 213 + 426 \cdot 2^{15}) \cdot 10.082.083}{2^{32}} \right\rfloor - 2^{15} = \left\lfloor \frac{S + 213}{426} \right\rfloor \tag{46}$$

2. **Bước 2 — Nhân cửa sổ Hanning:** Độ lệch $d[j] = x[j] - \text{mean}$ là số nguyên có dấu 17-bit (nằm trong khoảng $[-65.536, 65.535]$). Mẫu sau khi nhân cửa sổ Hanning định dạng Q1.15 được làm tròn và dịch phải:
$$xw[j] = (d[j] \cdot w[j] + 2^{14}) \gg 15 \tag{47}$$
với $w[j] \in [1, 32.767]$ đọc từ ROM. Khung biên cực đại có thể đạt $|xw| = 59.379$, vượt quá 16-bit nên yêu cầu thanh ghi 17-bit có dấu.

3. **Bước 3 — Đỉnh biên độ cục bộ (LP):** Tìm giá trị cực đại tuyệt đối của mẫu trong vùng trung tâm 214 mẫu quanh tâm khung:
$$LP = \max_{j=212}^{425} |xw[j]| \tag{48}$$

4. **Bước 4 — Khối nhân-cộng tự tương quan (MAC):** Tích lũy năng lượng khung $R[0] = \sum_{j=0}^{637} xw[j]^2$ và tương quan chập cho 189 độ trễ:
$$R[\tau] = \sum_{j=0}^{637-\tau} xw[j] \cdot xw[j+\tau], \quad \tau \in [26, 214] \tag{49}$$

5. **Bước 5 — Chuẩn hóa thang đo động:** Tìm số bit dịch động $sh = \max(0, \text{bitlen}(R[0]) - 26)$ để đưa $R[0]$ và toàn bộ các giá trị $R[\tau]$ về dải số nguyên 27-bit:
$$R_{0n} = R[0] \gg sh, \quad R_n[\tau] = R[\tau] \gg sh \tag{50}$$

6. **Bước 6 — Bù suy giảm cửa sổ và trừ chi phí quãng tám (Scoring):** Tích số chuẩn hóa $U[\tau]$ và điểm số đánh giá $S_c[\tau]$ được tính bằng phép nhân số nguyên với các hệ số đọc từ ROM:
$$U[\tau] = (R_n[\tau] \cdot C[\tau] + 2^{13}) \gg 14 \quad (\approx r(\tau) \cdot R_{0n}) \tag{51}$$
$$S_c[\tau] = U[\tau] - \big((R_{0n} \cdot K[\tau] + 2^{19}) \gg 20\big) \quad \left(\approx \left(r(\tau) - 0{,}01 \log_2 \frac{600}{f}\right) \cdot R_{0n}\right) \tag{52}$$
với $C[\tau] \in$ Q2.14 và $K[\tau] \in$ Q12.20.

7. **Bước 7 — Trích chọn ứng viên cực đại cục bộ:** Quét tìm các độ trễ $\tau \in [27, 213]$ thỏa mãn điều kiện tạo đỉnh cục bộ:
$$U[\tau] > U[\tau - 1], \quad U[\tau] \geq U[\tau + 1], \quad 1000 \cdot U[\tau] > 225 \cdot R_{0n} \tag{53}$$
Ứng viên tốt nhất được chọn là vị trí độ trễ τ_p có điểm số $S_c[\tau]$ lớn nhất (argmax).

8. **Bước 8 — Quyết định hữu thanh/vô thanh (Voicing):** Khung được xác nhận là hữu thanh khi và chỉ khi bất đẳng thức tích chéo số nguyên sau đây thỏa mãn:
$$(100 \cdot S_c[\tau_p] - 45 \cdot R_{0n}) \cdot 3 \cdot GP > 100 \cdot R_{0n} \cdot \max(0, 6 \cdot GP - 145 \cdot LP) \tag{54}$$
Bất đẳng thức này phản ánh chính xác điều kiện ngưỡng voicing của Praat: strength $> 0{,}45 + \max(0, 2 - I/(0{,}03/1{,}45))$ với cường độ tương đối $I = LP/GP$.

9. **Bước 9 — Nội suy đỉnh parabol và nghịch đảo tần số F0 gộp:** Lấy hiệu độ dốc $num = U[\tau_p{+}1] - U[\tau_p{-}1]$ và độ cong $den = 2U[\tau_p] - U[\tau_p - 1] - U[\tau_p + 1]$. Sau khi dịch chuẩn hóa $s_2 = \max(0, \text{bitlen}(den) - 24)$, mẫu số chung $D = 2 \cdot den_n \cdot \tau_p + num_n$. Giá trị tần số F0 định dạng Q12.4 được tính gọn bằng duy nhất một phép chia số nguyên:
$$F0_Q12.4 = \left\lfloor \frac{512.000 \cdot den_n + \lfloor D/2 \rfloor}{D} \right\rfloor \tag{55}$$

10. **Bước 10 — Bộ lọc trung vị median-3:** Đầu ra tức thời của khung là $F0_Q12.4$ nếu hữu thanh hoặc 0 nếu vô thanh; sau đó được đưa qua bộ lọc trung vị trượt 3 khung liên tiếp để triệt tiêu các xung nhảy quãng tám cô lập.

0.1.4 Kết quả đối chiếu trên tập tín hiệu tổng hợp chuẩn

Để đánh giá định lượng độ chính xác của bộ mô hình `golden_v2`, ta thực hiện kiểm thử toàn diện trên hai tệp âm thanh tổng hợp chuẩn (tần số lấy mẫu 16 kHz, độ dài 2,8 giây, gồm 277 khung phân tích, bước nhảy hop 160 mẫu, $t_1 = 0{,}020$ s). Tệp thứ nhất là tín hiệu sạch không tạp âm (`synth_glide_vuv_16k.wav`, $GP = 22.946$), tệp thứ hai được cộng thêm nhiễu trắng ở tỷ số $SNR = 5$ dB (`synth_glide_vuv_16k_snr5.wav`, $GP = 22.937$) để kích hoạt các tình huống nhảy quãng tám phức tạp. Kết quả tổng hợp từ công cụ `golden_v2/compare.py` được trình bày trong Bảng 4.

Bảng 4: Kết quả đối chiếu toàn tuyến bộ công cụ golden v2 với Praat 7.0.03 trên tín hiệu tổng hợp chuẩn (277 khung, 16 kHz).

Phép so sánh	**VDE**	**GPE**	**FFE**	**Cent TB**	**Cent p_{95}**	**Cent Max**
Tín hiệu sạch (Clean synth, 208 khung cùng hữu thanh):						
Port float vs Praat R2	0/277	0/208	0,0000	$5{,}7 \cdot 10^{-6}$	$5{,}6 \cdot 10^{-5}$	$6{,}8 \cdot 10^{-5}$
Port float vs Praat R1	0/277	0/208	0,0000	$5{,}7 \cdot 10^{-6}$	$5{,}6 \cdot 10^{-5}$	$6{,}8 \cdot 10^{-5}$
Praat R1 vs Praat R2	0/277	0/208	0,0000	0,00	0,00	0,00
Twin (trước median) vs Praat R2	0/277	0/208	0,0000	0,21	0,48	1,08
Twin + median-3 vs Praat R1	0/277	0/208	0,0000	0,37	0,64	12,96
Praat R1 vs Ground-Truth	4/277	0/207	0,0144	0,30	0,83	7,91
Twin + median-3 vs Ground-Truth	4/277	0/207	0,0144	0,53	1,05	14,28
Tín hiệu có ồn SNR 5 dB (207 khung cùng hữu thanh):						
Port float vs Praat R2	0/277	0/207	0,0000	$9{,}4 \cdot 10^{-6}$	$6{,}0 \cdot 10^{-5}$	$1{,}5 \cdot 10^{-4}$
Port float vs Praat R1	0/277	0/207	0,0000	$1{,}0 \cdot 10^{-5}$	$6{,}2 \cdot 10^{-5}$	$1{,}5 \cdot 10^{-4}$
Praat R1 vs Praat R2 (Viterbi)	0/277	19/207	0,0686	117,4	1191	37,28
Twin (trước median) vs Praat R2	0/277	3/207	0,0108	3,10	9,80	32,91
Twin + median-3 vs Praat R1	0/277	15/207	0,0542	3,60	10,90	32,91
Praat R1 vs Ground-Truth	3/277	0/207	0,0108	10,56	21,80	29,32
Praat R2 vs Ground-Truth	3/277	19/207	0,0794	126,4	1199	28,48
Twin (trước median) vs Ground-Truth	3/277	22/207	0,0903	142,3	1199	25,85
Twin + median-3 vs Ground-Truth	3/277	15/207	0,0650	94,99	1179	32,68

Đọc bảng. Bảng 4 làm sáng tỏ các mối quan hệ nền tảng của toàn bộ hệ thống:

- *Độ chính xác của bản port mã nguồn (Port float vs Praat):* Sai số cent giữa bản Python dấu phẩy động và phần mềm Praat 7.0.03 gốc chỉ ở mức 10^{-5} đến 10^{-4} cent, với $VDE = 0$ và $GPE = 0$. Con số cực nhỏ này khẳng định ta đã mô phỏng chuẩn xác 100% thuật toán C++ của Praat đến từng phép toán dấu phẩy động kép.
- *Vai trò của bộ dò đường Viterbi (Praat R1 vs R2):* Trên tín hiệu sạch, cấu hình mặc định R_1 và cấu hình không chi phí R_2 cho kết quả trùng khít 100%. Tuy nhiên, khi chịu ảnh hưởng của tạp âm mạnh (SNR 5 dB), bộ dò đường Viterbi của R_1 sửa được 19 khung nhảy quãng tám mà R_2 (thuần túy chọn đỉnh lớn nhất từng khung) mắc phải.
- *Độ chính xác của lõi số nguyên (Twin vs Praat R2):* Trên tín hiệu sạch, mô hình số nguyên khớp hoàn hảo với Praat R2: $VDE = 0$, $GPE = 0/208$, sai số cent mịn trung bình chỉ $0{,}21$ cent, $p_{95} = 0{,}48$ cent và sai số cực đại là $1{,}08$ cent (hoàn toàn không thể nhận biết bởi tai người). Trên tín hiệu ồn 5 dB, twin chỉ lệch 3 khung GPE so với R2 do sai số lượng tử hóa số nguyên ở các đỉnh có điểm số gần bằng nhau.
- *Hiệu quả của bộ lọc trung vị phần cứng (Twin + median-3 vs Ground-Truth):* Trên tín hiệu ồn 5 dB, so với sự thật mặt đất (ground-truth), bộ lọc median-3 giúp giảm số khung lỗi thô GPE của phần cứng từ 22 khung xuống còn 15 khung (sửa được 7 khung lỗi quãng tám đơn lẻ), chứng minh tính hiệu quả vượt bậc của một cấu trúc phần cứng siêu nhỏ gọn.

0.1.5 Ba khác biệt có chủ ý và phân tích hạn chế cố hữu

Để giữ sự minh bạch khoa học tuyệt đối, ta cần chỉ rõ ba sai khác có chủ ý giữa thiết kế phần cứng FPGA và thuật toán gốc của Praat (đã được thống nhất tại Mục 0.3):

1. **Nội suy đỉnh parabol thay cho sinc70 + Brent:** Trên tín hiệu sạch, parabol cho sai số nội suy nhỏ hơn $1{,}1$ cent. Tuy nhiên, trên tín hiệu có ồn lớn (SNR 5 dB), hình dạng đỉnh tự tương quan bị méo mó hoặc chẻ đôi, khiến xấp xỉ parabol lệch tới 32,9 cent so với tinh chỉnh sinc70. Đây là cái giá phải trả để tiết kiệm hàng chục nghìn chu kỳ tính toán và tránh thuật toán tìm nghiệm Brent phi tuyến tính trên FPGA.
2. **Xếp hạng ứng viên bằng điểm số trên lưới rời rạc:** Thuật toán phần cứng chọn đỉnh ứng viên tốt nhất $\tau_p = \text{argmax}\, S_c[\tau]$ dựa trên các giá trị nguyên rời rạc trên lưới trước khi nội suy, thay vì tính điểm sức mạnh S_c tại tọa độ thực sau nội suy như Praat. Khi hai đỉnh quãng tám có điểm số chênh lệch cực nhỏ (chỉ từ $0{,}0005$ đến $0{,}007$), việc làm tròn số nguyên có thể làm lật thứ tự ưu tiên (gây ra hiện tượng lệch 3/207 khung ở tệp thử nghiệm SNR 5 dB).

3. **Bộ lọc trung vị median-3 thay cho bộ dò đường quy hoạch động Viterbi:** Vì FPGA xử lý luồng theo từng khung thời gian thực và không lưu toàn bộ ma trận ứng viên của cả tệp âm thanh vào bộ nhớ lớn, nó không thể chạy thuật toán Viterbi toàn cục. Median-3 chỉ quan sát cửa sổ cục bộ 3 khung nên chỉ sửa được các lỗi nhảy quãng tám đơn lẻ (1 khung), không thể khắc phục được chuỗi nhảy quãng tám kéo dài từ 2 khung trở lên.

Ghi chú về tập dữ liệu thử nghiệm: Toàn bộ các con số kiểm chứng trong chương này được tính toán nghiêm ngặt trên tập tín hiệu tổng hợp chuẩn (`synth_glide_vuv_16k.wav` và phiên bản SNR 5 dB). Tập dữ liệu các tệp WAV giọng nói thực tế của anh Tú hiện chưa sẵn sàng tại thời điểm xuất bản chương này; khi có dữ liệu, kịch bản tự động `golden_v2/compare.py` sẽ được tái thực thi để cập nhật bổ sung mà không làm thay đổi kiến trúc kiểm chứng ba tầng đã thiết lập.

Tóm lại: Bản đồ kiểm chứng ba tầng bảo đảm sự nhất quán tuyệt đối giữa mã nguồn Python twin số nguyên, mô phỏng RTL, phần cứng thực tế trên kit DE10-Standard và chuẩn Praat 7.0.03; trên tín hiệu sạch, lõi phần cứng đạt sai số cent trung bình chỉ $0{,}21$ *cent và* $GPE = 0$*, còn trên tín hiệu ồn 5 dB, bộ lọc median-3 giúp giảm lỗi thô từ 22 khung xuống 15 khung.*